

UNIVENT – APLICAȚIE WEB PENTRU STUDENȚI

Candidat: Cătălin-Luca SECOȘAN

**Coordonatori științifici: Conf. dr.ing. Alexandru TOPÎRCEANU,
Ing. Darian VELCIOV**

Sesiunea: Iunie 2023

REZUMAT

Lucrarea de față urmărește proiectarea și dezvoltarea unei aplicații web, destinată utilizării de către studenți. Această platformă poartă denumirea de UNIVENT și le oferă tinerilor oportunitatea de a participa la evenimente extracurriculare și de a beneficia de experiențe unice pe perioada studiilor universitare. Astfel, această aplicație poate fi utilizată în mod gratuit de către orice student înrolat la una dintre universitățile partenere acestui proiect. În cadrul acestui site web, studenții au posibilitatea de a crea evenimente publice, de a se înrola ca participanți în evenimente create de alți studenți, de a-și vizualiza și edita informațiile profilului personal, de a vizualiza istoricul evenimentelor create și a participărilor la alte evenimente și de a oferi altor studenți din cadrul unui eveniment câte o recenzie.

Această aplicație a fost realizată cu ajutorul unor framework-uri și biblioteci speciale ale limbajelor de programare C# și JavaScript, mai exact ASP.NET Core și respectiv, React JS. De asemenea, au fost utilizate o serie de pachete adiționale pentru a facilita anumite operații, dar și API-uri gratuite, oferite de Google Cloud Platform. În plus, pentru a compila și executa codul, au fost folosite două medii de dezvoltare, pe 64 de biți, din suita Microsoft: Microsoft Visual Studio 2022 Community Edition și Microsoft Visual Studio Code.

Lucrarea prezentă este structurată în șapte capitole principale, ce vizează:

- O introducere în web;
- Stadiul actual al problematicei;
- Prezentarea unor fundamente teoretice și tehnologii utilizate;
- Proiectarea aplicației;
- Implementarea aplicației;
- Instrucțiuni de instalare și configurare, iar apoi un manual de utilizare;
- Concluzii și direcții de continuare a dezvoltării.

CUPRINS

1. INTRODUCERE	4
1.1 CONTEXT	4
1.2 MOTIVAȚIA LUCRĂRII	10
1.3 TEMATICA LUCRĂRII.....	11
2. STADIUL ACTUAL ÎN DOMENIUL PROBLEMEI	12
3. FUNDAMENTE TEORETICE ȘI TEHNOLOGII UTILIZATE	14
3.1 FUNDAMENTE TEORETICE.....	14
3.2 TEHNOLOGII UTILIZATE.....	15
3.2.1 ASP.NET CORE	15
3.2.2 MICROSOFT SQL SERVER	16
3.2.3 HTML	17
3.2.4 CSS	18
3.2.5 JAVASCRIPT	19
3.2.6 REACT JS	20
4. PROIECTAREA APLICAȚIEI.....	21
4.1 ANALIZA CERINȚELOR SISTEMULUI.....	21
4.1.1 CERINȚE FUNCȚIONALE.....	21
4.1.2 CERINȚE NON-FUNCȚIONALE	24
4.2 CAZURI DE UTILIZARE	25
4.2.1 ACTORII SISTEMULUI	25
5. IMPLEMENTAREA APLICAȚIEI	30
5.1 ARHITECTURA SISTEMULUI.....	30
5.2 PARTEA DE SERVER.....	30
5.3 BAZA DE DATE.....	42
5.4 CLIENTUL WEB	44
6. CONFIGURARE ȘI UTILIZARE	47
6.1 INSTALARE ȘI CONFIGURARE.....	47
6.2 MANUAL DE UTILIZARE	47
6.2.1 FUNCȚIONALITĂȚI STUDENT	52
6.2.2 FUNCȚIONALITĂȚI ADMINISTRATOR.....	62
7. CONCLUZII ȘI DIRECȚII DE CONTINUARE A DEZVOLTĂRII	66
REFERINȚE BIBLIOGRAFICE	67

1. INTRODUCERE

1.1 CONTEXT

Tehnologia Informației reprezintă una dintre cele mai dinamice industrii ale lumii și implică procurarea, prelucrarea, stocarea și transferul informației, în special pe calea digitală, cu ajutorul calculatoarelor electronice. Această industrie se află într-un proces constant de schimbare și evoluție, iar principalele laturi ale sale cuprind dezvoltarea software, hardware, a rețelelor de comunicație și multe altele.

Dezvoltarea software a fost de-a lungul timpului, una dintre laturile principale ale domeniului IT care a revoluționat la nivel global. Aceasta a fost mereu caracterizată prin introducerea și expansiunea diverselor limbaje de programare, menite să faciliteze oferirea de soluții digitale, în scopul satisfacerii nevoilor societății și ale afacerilor. În acest fel, dezvoltarea software a devenit rapid o parte esențială a aproape fiecărei industrii dar și un pilon important în susținerea economiei globale. Spre exemplu, datorită atât inovațiilor, cât și ale oportunităților de carieră aduse de această industrie, numărul de job-uri disponibile a crescut considerabil în ultima perioadă, carierele în acest domeniu fiind mai solicitate ca niciodată.

Cu toate acestea, ideea de internet provine cu mult înainte de era informației digitale, mai exact din anii 1890. Nikola Tesla, fizician, inginer și unul dintre cei mai renumiți oameni de știință, a propus ideea de a conecta lumea prin intermediul unui sistem numit „world wireless”. Ideile lui Tesla pentru a proiecta un asemenea sistem au fost inspirate de experimentele lui Hertz cu unde electromagnetice și transformatoare de bobină de inducție. Reproducând aceste experimente, Tesla a reușit mai târziu să își creeze propriul transmițător wireless. Astfel, după această reușită, planul lui Nikola Tesla ar fi fost construirea unei stații pentru a transmite mesaje peste Atlantic, prin energie fără fir. În cele din urmă s-a renunțat la acest concept în 1906, ideea de a transmite informații prin intermediul wireless fiind abandonată [1].

Odată cu apariția și evoluția calculatoarelor, noțiunea de transmitere fără fir a informației a fost revizitată, iar în 1960 Joseph Carl Robnett Licklider, directorul Biroului Tehnic de Prelucrare a Informațiilor, Pentagon, a propus ideea de rețea electronică, descrisă ca „principalul și esențialul mediu de interacțiune informațională pentru guverne, instituții, corporații și persoane fizice”. Această rețea avea să se numească „Rețeaua Intergalactică” și să fie accesibilă tuturor. Mai apoi, spre sfârșitul anilor 1960, a fost utilizat noul concept la vremea respectivă (cel de a transmite datele prin comutarea pachetelor), în apariția rețelei ARPANET. Această rețea a fost implementată de Agenția pentru Proiecte de Cercetare Avansată în Domeniul Apărării, SUA (prescurtat DARPA – „Defence Advanced Research Projects Agency”) și era utilizată pentru stabili o comunicare între mai multe calculatoare. Mai mult de atât, peste câțiva ani, această rețea a fost îmbunătățită prin apariția protocolului TCP/IP, pentru comunicarea datelor, având la bază modelul „client-server”. Succesul acestei rețele reprezintă un pas foarte important în evoluția internetului, deoarece mulți consideră ARPANET ca fiind strămoșul Internetului, așa cum îl cunoaștem astăzi [2].

În anul 1979, DARPA a împărțit rețeaua ARPANET în două rețele: o rețea pentru uz comercial și universitar, iar cealaltă pentru uz militar. Întrucât aceste două rețele puteau în continuare să comunice între ele, s-a creat o „inter-rețea”, ce poartă numele de Internet. Această mutare a atras atenția cercetătorilor, care au început să dezvolte cât mai multe aplicații de comunicare, bazate pe protocoale „bine definite” [2].

La zece ani după separarea rețelei ARPANET, Tim Berners-Lee, programator și cercetător britanic la Centrul European pentru Fizica Nucleară din Geneva (CERN), a lansat primul prototip World Wide Web. În cadrul CERN, Tim a întâmpinat dificultăți în găsirea informațiilor necesare, datorită organizării acestora în diferitele calculatoare ale centrului european, calculatoare ce foloseau diverse credențiale pentru logare. În acest mod, a fost propusă o metodă de a partaja și organiza informații provenite de la diverse sisteme informatice, în librării diferite, în funcție de localizarea geografică. Termenul World Wide Web a fost ales drept nume pentru această soluție, deoarece descrie cu adevărat formatul global și descentralizat al Internetului, adică o structură asemănătoare cu forma pânzei de păianjen [3].

În ziua de azi, sistemul WWW are la bază principiul de Hyperlinks, ce reprezintă legături și referințe textuale, utilizate pentru a naviga printre anumite secțiuni de informație de pe Internet, stocate în librăriile serverelor utilizate. Odată executat un hyperlink, destinația acestei referințe este apelată imediat, stabilindu-se conexiunea către conținutul ales. În plus, standardul folosit de WWW pentru a gestiona hyperlink-uri este Hypertext Transfer Protocol (HTTP). Acest protocol este caracterizat prin schimbul de date dintre un client și un server, utilizând un flux continuu de mesaje [4]. De cele mai multe ori, rolul de client îi aparține navigatorului web (în engleză „browser”), el fiind cel care interpretează paginile web create în diverse limbaje de programare și afișează utilizatorului conținutul într-un format lizibil.

Mesajele folosite în comunicarea dintre client și server poartă denumirea de cereri HTTP (în engleză „HTTP requests”), iar aceste solicitări sunt alcătuite din trei componente principale:

- Linia de start. Aceasta conține metoda cererii HTTP, o adresă URI (Uniform Resource Identifier) sau o cale absolută către protocolul, portul și domeniul resursei țintă, iar apoi linia se încheie versiunea protocolului HTTP. Metoda cererii indică tipul de acțiune pe care clientul dorește să o efectueze asupra resursei și reprezintă un verb în limba engleză. Cele mai populare metode sunt „GET”, „POST”, „PUT”, „DELETE” și „PATCH”. De exemplu, „GET” este folosit pentru a obține o resursă, „POST” pentru a trimite date noi către server, „PUT” pentru a înlocui date existente, cu altele noi, „DELETE” pentru a șterge resurse și „PATCH” pentru a actualiza parțial resurse existente. Nu în ultimul rând, două exemple ilustrative pentru o linie de start sunt: „GET /background.png HTTP/1.0” și „GET https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages HTTP/1.1” [5].
- Anteturi (numite „Headers” în engleză). Acestea conțin informații suplimentare despre cerere, precum tipul de conținut acceptat de client, limbajul preferat, informații despre autentificare, informații despre browser și multe altele. Fiecare

antet conține o singură linie și ajută serverul să înțeleagă și să proceseze corect cererea. Structura utilizată pentru antet este cea a perechilor cheie-valoare, fiecare linie cuprinzând un șir de caractere pentru numele antetului, apoi două puncte și, în final, valoarea propriu-zisă. În general, anteturile pot fi de mai multe tipuri: anteturi generale (utilizate pentru mesaje), anteturi de cerere (utilizate pentru a modifica cererea, oferind sau restricționând context adițional), sau anteturi de reprezentare (utilizate pentru a descrie datele mesajului și codificarea aplicată acestuia) [5].

- Corpul cererii, ce este o componentă opțională. Corpul poate conține date suplimentare care urmează a fi trimise către server. Dacă este prezent, corpul cererii poate conține o singură resursă, pentru un singur fișier, sau mai multe resurse, cu informații diferite [5].

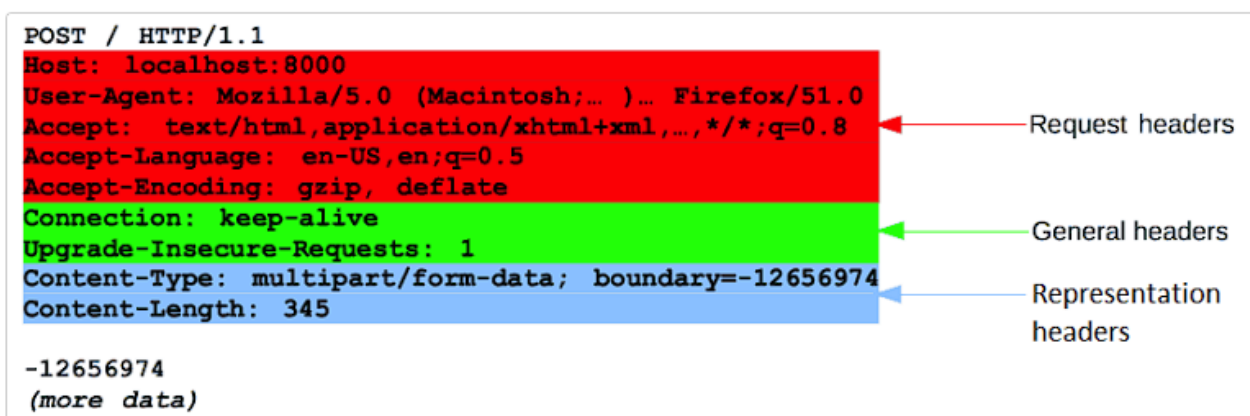


Figura 1.1: Exemplu de cerere HTTP [5]

În cazul răspunsurilor primite din partea serverului, ele respectă aceeași structură ca și cea a cererilor, cu mici diferențe:

- Prima linie se numește linie de stare acum și include, în ordinea menționată, versiunea protocolului HTTP, un cod de stare (pentru a indica succesul sau eșecul cererii primite de la client) și o descriere de stare, pentru a oferi mai multe informații legate de rezultatul obținut în urma execuției cererii. Un exemplu de o asemenea linie este: „HTTP/1.1 404 Not Found” [5].
- Anteturile rămân în principiu la fel, însă pot apărea în altă ordine de această dată [5].
- Corpul răspunsului poate include fișiere atât de dimensiune cunoscută ca și în cazul cererilor HTTP, cât și fișiere de dimensiune necunoscută, care necesită a fi gestionate diferit [5].

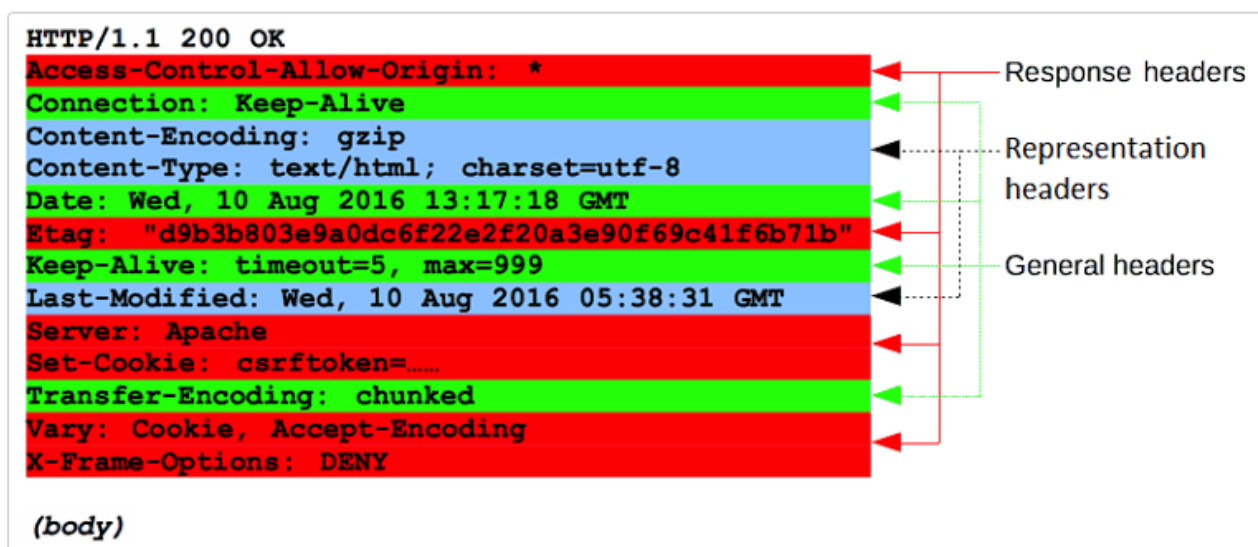


Figura 1.2: Exemplu de răspuns HTTP [5]

Codurile de stare ale răspunsurilor HTTP sunt clasificate în cinci categorii diferite, în funcție de situație:

- 1xx. Această clasă este utilizată pentru erori informaționale, drept răspunsuri provizorii ale serverului. Cele mai întâlnite cazuri sunt „100 - Continuă” (utilizat pentru a indica utilizatorului faptul că cererea nu a fost respinsă încă și acesta poate continua) și „101 - Schimbare de Protocol” (anunță clientul că serverul va îndeplini cererea dar părți ale acestuia se află în proces de schimbare sau mutare). [6]
- 2xx. Codurile din categoria 200 sunt folosite pentru a indica clientului faptul că cererea a fost acceptată și executată cu succes. Câteva exemple sunt „200 - Ok” (mesaj simplu pentru a notifica utilizatorul că cererea a fost efectuată cu succes), „201 - Creat” (seamănă cu răspunsul precedent, dar rezultatul a fost o resursă nou creată pe server), „202 - Acceptat” (indică faptul că cererea a fost acceptată pentru a fi procesată, însă procesul nu a fost finalizat încă), „203 - Informații Non-Autoritative” (cererea a fost primită și procesată, dar conținutul returnat provine de la server secundar, nu de la sursa principală) sau „204 - Fără Conținut” (de această dată, cererea a fost executată cu succes, dar serverul nu returnează niciun conținut în răspuns). [6]
- 3xx, coduri folosite pentru redirecționări. De cele mai multe ori, clientul este redirecționat către o nouă adresă, ce o preia dintr-un antet al răspunsului. Aici avem „300 - Opțiuni Multiple” (returnat atunci când serverul are mai multe opțiuni disponibile pentru resursa cerută și solicită clientului să aleagă una dintre aceste opțiuni; răspunsul conține o listă de adrese alternative sau informații suplimentare pentru a ghida clientul în luarea deciziei corecte), „301 - Mutat Permanent” (în acest caz, resursa cerută a fost mutată permanent la o altă locație), „302 - Găsit” (resursa țintă a fost mutată temporar la o altă locație), „303 - Vezi Altă Sursă” (foarte asemănător cu cazul 302, însă aici clientul ar trebui să

- efectueze o nouă cerere GET către noua locație specificată), „304 - Nemodificat” (folosit în situațiile în care clientul realizează o cerere de tip GET sau HEAD și resursa solicitată nu s-a modificat de la ultima cerere) și altele. [6]
- 4xx. Clasa 400 este formată din coduri care sunt des întâlnite de utilizatori, indicând erori ale acestora. Spre exemplu, avem „400 - Cerere Greșită” (serverul nu poate procesa cererea clientului din cauza unei erori de sintaxă sau a unei cereri invalide), „401 - Neautorizat” (cererea clientului nu poate fi procesată de server, deoarece accesul la resursă este restricționat, sau cererea necesită autentificare în prealabil), „402 - Necesită Plată” (acest cod de stare este special rezervat pentru cazurile în care accesul la resursă este condiționat de efectuarea unei plăți), „403 - Interzis” (serverul a înțeles cererea, însă refuză să ofere acces la resursa solicitată de către client), „404 - Negăsit” (resursa cerută nu a fost găsită) și altele. [6]
 - 5xx. Această categorie indică utilizatorului că au existat erori ale serverului, în execuția cererii. Câteva cazuri sunt: „500 - Eroare Internă de Server” (a întâmpinat o eroare neașteptată, care îl împiedică să proceseze și să finalizeze cererea clientului), „501 - Nu este Implementat” (utilizat atunci când serverul nu poate gestiona, sau nu suportă funcționalitatea solicitată în cadrul cererii), „502 - Poartă Greșită” (cel mai întâlnit atunci când serverul acționează ca o poartă sau un proxy între alte servere și primește un răspuns invalid de la un astfel de server intermediar), „503 - Serviciu Indisponibil” (în acel moment, serverul nu poate executa cererea primită datorită unor supraîncărcări sau incapacități temporare) sau „504 – Expirarea Timpului de Așteptare la Poartă” (serverul care acționează ca intermediar, nu primește un răspuns de la serverul final în intervalul de timp specificat). [6]

Anterior, am menționat faptul că WWW utilizează referințe de tip hyperlink, pentru a ajunge la conținutul cerut și a efectua diverse operațiuni. O asemenea adresă completă a unei pagini sau fișier de pe internet poartă numele de Uniform Resource Locator (URL). Cu toții am văzut aceste adrese unice în momentul în care am căutat informații pe internet, în bara de adresă a motoarelor de căutare, poziționată de cele mai multe ori în partea de sus a ecranului. Conceptul de URL este foarte asemănător cu căile absolute ale fișierelor de pe calculator, însă au o structură diferită:

- La început se găsește un acronim pentru denumirea protocolului. Dacă acest segment nu este specificat, în mod implicit este ales protocolul HTTP (alte protocoale sunt: HTTPS, FTP, NTP, etc.) [7].
- Subdomeniul, este o parte opțională a adresei URL, aflată înainte de domeniul principal și ajută la organizarea și separarea secțiunilor unui site web [7].
- Domeniul, care reprezintă numele propriu-zis al website-ului, fiind adresa principală a resursei online pentru care se dorește accesarea [7].
- Domeniul de nivel superior, este ultima parte a unei adrese URL, care indică categoria sau tipul de entitate asociată cu un website sau o resursă online. Acesta este cunoscut și sub numele de extensie de domeniu, fiind alcătuit din

una sau mai multe litere. Domeniile de nivel superior pot fi de diferite tipuri, fiecare având o semnificație specifică [7]. Unele exemple comune, ar fi: „.com” (utilizat pentru site-uri comerciale, sau afaceri), „.org” (asociat organizațiilor non-profit), „.gov” (pagini web ale agențiilor guvernamentale), „.net” (destinate site-urilor legate de infrastructuri de rețea), „.ro” (extensie rezervată pentru site-urile din România), „.edu” (utilizat de instituții educaționale, de cele mai multe ori școli sau universități) [8].

- Subfolder: prima secțiune amplasată după domeniu și este specifică fiecărui website în parte. Este utilizat pentru a organiza și structura conținutul disponibil, în diferite categorii sau secțiuni. Un bun exemplu ar fi „nume_site.ro/blog”, unde „blog” reprezintă subfolderul dedicat articolelor de blog [7].
- Slug: este partea adresei URL care identifică în mod unic o anumită pagină sau resursă din cadrul întregului website. Acesta este adesea utilizat pentru a crea adrese URL descriptive și ușor de înțeles pentru utilizatori. De exemplu, putem avea „nume_site.ro/blog/evolutia-calculatoarelor”, unde „evolutia-calculatoarelor” reprezintă slug-ul adresei [7].
- Ancora. Acest element permite utilizatorului să sară în mod direct, la o anumită secțiune sau locație specifică de pe o pagină web, fără a fi nevoie să deruleze întregul conținut. Acest comportament este realizat de obicei, prin utilizarea unei referințe interne [7].

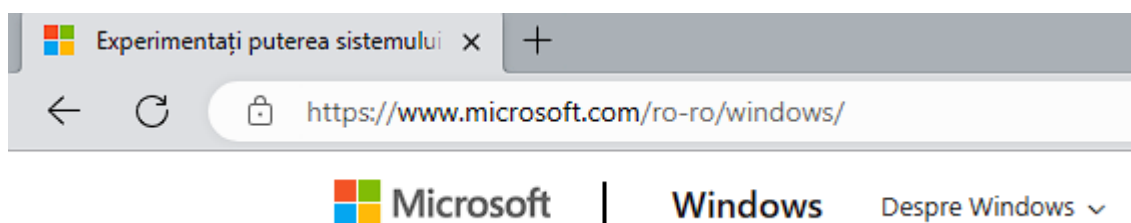


Figura 1.3: Exemplu de adresă completă URL

În exemplul de mai sus, protocolul este „https” (pentru transfer hipertext securizat), subdomeniul este „www”, domeniul este „microsoft” (site-ul ales ca exemplu aparținând companiei Microsoft), extensia acestuia este „.com” (pagină web comercială) și subfolderul utilizat este „ro-ro” (o parte a întregului site, dedicată regiunii România), cu slug-ul „windows” (pentru o secțiune specifică, despre Windows).

Pentru a adăuga securitate în transmiterea datelor prin cereri și răspunsuri HTTP, a apărut, mai târziu, protocolul HTTPS (Hypertext Transfer Protocol Secure), ce funcționează similar cu versiunea nesecurizată. Deoarece HTTP nu includea mecanisme de criptare a datelor, informațiile transmise erau expuse riscului de interceptare prin acces neautorizat. Nevoia unui protocol securizat pentru schimbul de date cu caracter confidențial, precum autentificare în cont, date personale sau detalii de plată, a condus la apariția HTTPS. Această îmbunătățire permite securizarea printr-un protocol ce utilizează chei criptografice atât pentru a cripta date, cât și pentru a le valida. Cele mai utilizate metode de securizare a unui domeniu îl reprezintă obținerea anumitor certificate, precum SSL („Secure

Sockets Layer”), sau TLS („Transport Layer Security”). TLS este o versiune actualizată a lui SSL, ce rezolvă câteva vulnerabilități de securitate, însă, la ora actuală, ambele sunt acceptate. De asemenea, utilizatorii pot verifica dacă website-ul pe care aceștia navighează cuprinde certificate pentru securitate foarte ușor. Cele mai multe navigatoare folosesc o iconiță specială, de lacăt, dacă protocolul utilizat de site este HTTPS. În caz contrar, poate apărea simbolul „!” sau eticheta „Nesecurizat” [9].

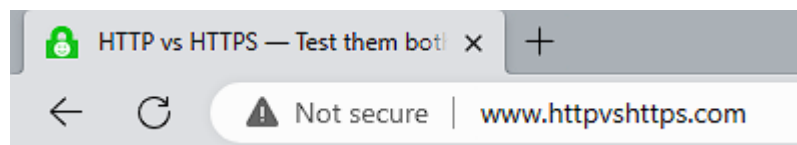


Figura 1.4: Exemplu de site nesecurizat

1.2 MOTIVAȚIA LUCRĂRII

Încă de pe vremea Greciei Antice, filosoful Aristotel a definit omul ca fiind o ființă socială, spunându-se astfel că o viață fără interacțiuni interumane ar putea duce la apariția aspectelor negative în comportamentul și dezvoltarea personală. Astfel, interacțiunea socială joacă un rol crucial în formarea identității și în procesul de învățare și creștere personală. Prin interacțiunea cu ceilalți, oamenii își pot împărtăși idei, experiențe și emoții, pot învăța de la ceilalți și pot dezvolta abilități de comunicare și relaționare. De asemenea, interacțiunile sociale contribuie și la construirea rețelelor sociale, la crearea legăturilor emoționale sau la susținerea reciprocă în fața dificultăților, acestea devenind esențiale pentru bunăstarea și dezvoltarea armonioasă a individului în cadrul societății. Acest aspect este valabil și în rândul tinerilor, în special pentru cei care se află în perioada de descoperire a propriei identități.

În primul rând, prin interacțiunea cu alte persoane, tinerii își pot lărgi perspectiva asupra lumii, învățând despre diversitatea culturală, valorile diferite, modurile diferite de gândire și abordări vaste în rezolvarea problemelor. Prin interacțiuni cu alți tineri, ce au experiențe și povești de viață diferite, studenții își pot dezvolta sentimentele de empatie și înțelegere față de ceilalți, ceea ce le va fi de folos pe termen lung, atât în viața personală, cât și pe plan profesional.

Socializarea oferă tinerilor oportunități nemăsurate, una dintre cele mai importante fiind clădirea cu succes a relațiilor sociale, dezvoltând prietenii solide. Aceste conexiuni asigură și un sprijin emoțional, iar prin participarea la evenimente extracurriculare, studenții pot descoperi noi interese sau oportunități în diverse domenii.

Un alt aspect important, oferit de relaționarea cu persoane de vârstă apropiată, este capacitatea de a comunica și de a asculta persoanele din jur. Socializarea facilitează dezvoltarea abilităților de exprimare a propriilor idei, opinii sau puncte de vedere, dar și de a asculta și a înțelege perspectivele altora. Mai mult de atât, în timpul conversațiilor, tinerii își pot dezvolta abilități de negociere, de rezolvare a conflictelor și de lucru în echipă. Prin discuții, activități comune și participarea la evenimente sociale, tinerii își pot îmbogăți

cunoștințele și experiențele lor. Ei pot afla despre pasiunile și interesele altora, pot descoperi hobby-uri și activități noi și pot dobândi încredere în sine, a propriilor abilități. În acest fel, evenimentele sociale oferă oportunități inedite tinerilor, de a împărtăși experiențe și de a învăța unii de la ceilalți.

Așadar, am ales această temă pentru a oferi studenților din cadrul mai multor universități, șansa de a participa în mod gratuit la evenimente extracurriculare și de a beneficia de o experiență unică și completă, în perioada studiilor universitare. Pe lângă acestea, aplicația UNIVENT are și un rol important în procesul de acomodare al studenților în mediul universitar. Începutul vieții universitare poate fi o perioadă plină de provocări și adaptări pentru tineri, mai ales atunci când se confruntă cu un nou mediu, colegi noi și oportunități variate.

1.3 TEMATICA LUCRĂRII

Platforma UNIVENT este destinată tuturor studenților din cadrul universităților partenere și le pune acestora la dispoziție o serie de evenimente extracurriculare. Administratorul de sistem va adăuga în baza de date a aplicației o listă de tipuri de evenimente acceptate, în funcție de acordul stabilit cu universitățile partenere. Studenții vor putea să își creeze un cont gratuit, ce va fi activat după ce administratorul verifică împreună cu universitatea corespunzătoare, informațiile studentului. Odată înscris pe platformă, **studentul** va putea:

- **Să creeze un eveniment public.** Studentul care creează un eveniment va alege tipul de eveniment din listă, iar mai apoi își asumă rolul de gazdă sau îndrumător pentru restul participanților și se va asigura că totul va decurge bine, luând măsuri pentru prevenirea oricăror incidente nedorite. Totodată, studentul este responsabil și de actualizarea detaliilor evenimentului, în cazul schimbărilor de locație, amânare sau ajustare a numărului de participanți. Nu în ultimul rând, acesta are dreptul de a anula un eveniment, oricând înaintea începerii.
- **Să se înroleze în evenimente create de alți studenți.** Orice student conectat pe platformă va putea vizualiza evenimentele planificate și se va putea înrola drept participant la orice eveniment, în limita locurilor disponibile.
- **Să editeze informațiile de profil.** În cazul schimbării informațiilor personale, studentul își poate edita profilul de pe platformă. De asemenea, studenții pot folosi poze pentru profil, însă acest lucru este optional.
- **Să vizualizeze istoricul tuturor evenimentelor create și a evenimentelor în care acesta a participat.**
- **Să ofere recenzii studentului gazdă și restul participanților, după încheierea unui eveniment.**

2. STADIUL ACTUAL ÎN DOMENIUL PROBLEMEI

În ultimele două decenii, dezvoltarea tehnologică a deschis noi oportunități de socializare prin intermediul rețelelor de socializare virtuală, numite „social media”. Aceste aplicații au devenit extrem de populare și au transformat modul în care oamenii interacționează și se conectează în mediul online, numărul maxim de utilizatori fiind atins în timpul pandemiei COVID-19. În acest capitol, vom expune câteva dintre cele mai cunoscute aplicații de socializare virtuală și vom analiza caracteristicile lor distinctive și impactul pe care îl au în societate.

Cea mai populară rețea de socializare virtuală este Facebook, deținută de Mark Zuckerberg. La vremea respectivă, această platformă a revoluționat modul în care oamenii împărtășesc informații de la distanță și interacționează cu conținutul digital. Una dintre cele mai vechi funcționalități se numește „Events”, disponibilă încă din 2005, însă aceasta nu a stârnit mare interes până în 2011, când dezvoltatorii de la Facebook au introdus sugestii bazate pe locația curentă [10].

Un alt website, destinat planificării evenimentelor în timpul liber, este Meetup. Această aplicație are ca rol principal conectarea persoanelor ce împărtășesc interese comune, ajutându-i să organizeze simplu și rapid întâlniri sau evenimente într-o comunitate locală. Utilizatorii pot găsi diverse grupuri și se pot înscrie în evenimente, ce vor oferi experiențe din arii vaste de activitate. Pe de altă parte, platforma Ticketmaster este bine cunoscută pentru serviciile de cumpărare a biletelor digitale la categorii multiple de evenimente, precum concerte, spectacole de teatru, evenimente sportive și altele. Utilizatorii au posibilitatea de a căuta evenimente și de a gestiona rezervările curente, din secțiunea corespunzătoare profilului personal.

Pentru a evidenția mai bine opțiunile și serviciile oferite de platformele prezentate în cadrul acestui capitol, putem analiza tabelul de mai jos, în vederea identificării diferențelor.

Tabel 2.1: Comparație cu alte aplicații similare

Caracteristici	Facebook https://www.facebook.com/	Meetup https://www.meetup.com/	Ticketmaster https://www.ticketmaster.com/	UNIVENT
Aplicație web	Da	Da	Da	Da
Oferă client mobil	Da	Da	Da	Nu
Disponibilă Gratuit	Da	Da	Da	Da
Sugerare automată de evenimente în funcție de locația utilizatorului	Da	Da	Da	Nu

Orice utilizator poate crea un eveniment	Da	Da	Nu	Da
Orice utilizator se poate alătura unui eveniment	Da	Da	Această platformă vinde bilete pentru evenimente deja planificate, acționând ca un intermediar între organizatori și utilizatori	Da
Opțiunea de a te alătura unui grup	Reprezintă o funcționalitate diferită de „Events”	Da	Nu	Nu
Suportă categorii diferite de evenimente	Da	Da	Da	Da
Opțiunea de a acorda recenzii	Da	Da	Da	Da
Aplicație destinată exclusiv studenților	Nu	Nu	Nu	Da

3. FUNDAMENTE TEORETICE ȘI TEHNOLOGII UTILIZATE

3.1 FUNDAMENTE TEORETICE

Un site web are o structură mai complicată, care constă din multiple părți esențiale, ele lucrând împreună pentru a crea o experiență interactivă și cât mai plăcută pentru utilizator. Partea de client, partea de server și serverul bazei de date sunt toate componente care joacă un rol important în funcționarea și transmiterea conținutului dorit.

Partea de client (în engleză numită „Front-End”, prescurtat ca FE) este ceea ce vede utilizatorul final, atunci când vizitează un site web. Această componentă include paginile web și elementele de interfață grafică, definite în fișiere HTML, stilurile CSS și scripturile JavaScript (prescurtat JS). Partea de client rulează direct în navigatorul web al utilizatorului și este responsabilă atât pentru modul în care conținutul este afișat, cât și pentru interacțiunea și experiența utilizatorului. Prin intermediul browser-ului, clientul trimite cereri către server pentru a obține informațiile și resursele necesare pentru afișarea paginilor website-ului. De asemenea, pot exista mai multe aplicații client conectate la același server, simultan. Un exemplu ilustrativ ar fi o pagină web (accesibilă de pe un calculator sau un telefon mobil), ce oferă și o aplicație mobilă, ambele comunicând cu același server.

Cea de-a doua componentă principală din infrastructura unui site gestionează cererile clientului și le procesează. Aceasta poartă numele de server (în engleză „Back-End”, prescurtat ca BE) și conține logica de preluare, afișare și manipulare a datelor, precum și gestionarea funcționalităților specifice ale site-ului. Serverul interpretează solicitările HTTP primite din partea clientului, pentru a returna răspunsuri utilizate în pagini web create dinamic. Aceste răspunsuri cuprind date despre anumite entități, fișiere statice (de exemplu: documente sau imagini) sau alte resurse pe care clienții le solicită. Astfel, serverul asigură mereu că interacțiunea cu clienții se desfășoară fără probleme și că resursele solicitate sunt livrate într-un mod cât mai eficient.

Serverul pentru baza de date se ocupă cu stocarea și gestionarea tuturor datelor aplicației, aflându-se în strânsă legătură cu serverul BE. În cazul solicitărilor primite din partea unui client, acest server interacționează cu serverul bazei de date pentru a accesa și a modifica datele solicitate de client. În momentul în care un client a inițiat o cerere, partea de Back-End se conectează la serverul bazei de date pentru a manipula datele relevante, în funcție de acțiunea solicitată. Această conexiune facilitează utilizarea unor interogări sau comenzi specifice către baza de date. Primind aceste comenzi, serverul bazei de date le interpretează și apoi execută acțiunile adecvate. După ce acesta termină de executat sarcinile specificate, serverul bazei de date transmite rezultatele către serverul backend. Apoi, serverul backend procesează aceste constatări și creează răspunsul care este distribuit, mai departe, clientului. Răspunsul poate include datele solicitate sau un mesaj de confirmare sau eroare, în funcție de succesul sau eșecul operațiunii. În acest fel, se poate observa că partea Back-End reprezintă doar un intermediar între cererile lansate de client și baza de date propriu-zisă. Configurarea corectă și securizarea serverului BE garantează

astfel că datele sunt accesate și actualizate în mod corespunzător, în funcție de nevoile utilizatorilor.

Pentru a vizualiza mai ușor întregul proces, am atașat mai jos o diagramă, cu felul în care informațiile circulă între componentele unui website.

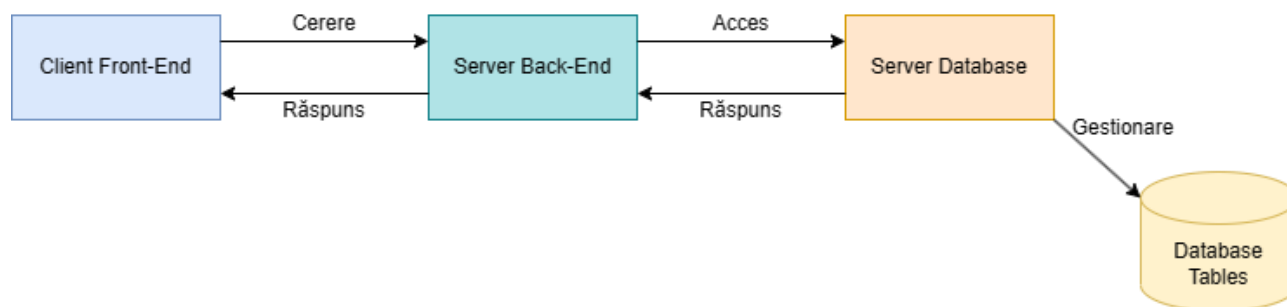


Figura 3.1: Schema componentelor unui site web

3.2 TEHNOLOGII UTILIZATE

În această secțiune vom prezenta detalii despre tehnologiile utilizate în dezvoltarea platformei UNIVENT.

3.2.1 ASP.NET CORE

ASP.NET Core reprezintă un framework open-source pentru dezvoltarea de aplicații web și alte servicii și a fost dezvoltat de către Microsoft, devenind o versiune îmbunătățită și modernizată a platformei ASP.NET deja existente. Programatorii pot crea aplicații web de înaltă calitate cu ASP.NET Core, care oferă un mediu de dezvoltare scalabil, performant și flexibil. Acest mediu versatil este bazat pe limbajul C# și permite implementarea aplicațiilor pe o varietate de platforme, cum ar fi Windows, Linux și macOS [11].

În același timp, ASP.NET Core oferă suport nativ pentru construirea API-urilor de tip RESTful. Stilul arhitectural REST (Representational State Transfer) permite o comunicare eficientă între client și server, prin utilizarea operațiunilor HTTP standard, pentru a accesa și gestiona resursele din baza de date [12].

Unul dintre aspectele esențiale ale framework-ului ASP.NET Core este abordarea sa bazată pe modularitate și componentizare. Framework-ul este împărțit în mai multe module și pachete, care pot fi utilizate în funcție de cerințele fiecărei aplicații. Acest lucru le permite dezvoltatorilor flexibilitate crescută, ușurință în gestionarea dependențelor și le oferă acestora șansa de a personaliza aplicația în funcție de cerințele lor [11].

Platforma ASP.NET Core a fost construită în jurul conceptului de middleware, ce permite configurarea și gestionarea modulară a fluxului de cereri HTTP și a răspunsurilor asociate, prin intermediul unui pipeline personalizat. În plus, adăugarea unui astfel de pipeline configurat le permite dezvoltatorilor să definească, în detaliu, fluxul de procesare al

cererilor primite și să adauge caracteristici suplimentare, cum ar fi: autentificare, autorizare, gestionare de erori cu mesaje suprascrise și multe altele [11].

Pe de altă parte, o trăsătură suplimentară a acestui framework este performanța ridicată, ce oferă o experiență de rulare rapidă și eficientă, datorită optimizărilor la nivel de bază, împreună cu o arhitectură modernă. Totodată, ASP.NET Core este capabil să suporte încărcarea și descărcarea componentelor în timp real, precum și actualizarea aplicației în timp real, fără a necesita restartarea serverului. Acest framework pune la dispoziție și o gamă largă de biblioteci sau instrumente, care facilitează dezvoltarea de aplicații web. Pachetele adiționale includ un sistem robust de rutare, mecanisme sofisticate de gestionare a dependențelor, suport pentru autentificare și autorizare, mapări relaționale ale obiectelor (ORM-uri) pentru interacțiunea cu bazele de date, suport pentru testare și multe altele. Mai mult de atât, această platformă dispune de o comunitate activă și o documentație bogată pentru dezvoltatorii software [11].

În comparație cu versiunile anterioare, ASP.NET Core 6 a adus o serie de îmbunătățiri semnificative, precum:

- Optimizări ale performanței, legate de viteza de pornire a aplicației, reducerea consumului de memorie și timp de răspuns mai rapid al aplicației;
- Integrare mai strânsă cu serviciile cloud: această versiune include suport îmbunătățit pentru Microsoft Azure, Amazon AWS și Google Cloud;
- Suport nativ pentru protocolul Remote Procedure Call (gRPC). Această funcționalitate le permite programatorilor să construiască aplicații de înaltă performanță, ce pot fi scalate;
- Suport pentru Blazor, framework ce utilizează C# și .NET pentru implementarea interfețelor interactive, dinamice și ușor de menținut;
- Suport pentru framework-ul .NET Multi-Platform App UI. Prescurtat ca și MAUI, această platformă facilitează construirea aplicațiilor mobile, native pentru sistemele iOS, Android și altele [11].

3.2.2 MICROSOFT SQL SERVER

SQL (Structured Query Language) este un limbaj de interogare structurat și standardizat, folosit pentru gestionarea bazelor de date relaționale. Datele stocate în bazele de date relaționale sunt salvate în tabele diferite, acestea devenind colecții de înregistrări, dispuse pe rânduri și coloane. Astfel, SQL permite crearea, interogarea, modificarea și gestionarea datelor și tabelelor. Acest limbaj face parte din standardul ANSI (American National Standards Institute), iar fiecare versiune suportă un set minim de comenzi, pentru a se conforma standardului. Cele mai populare comenzi sunt „SELECT”, „INSERT”, „UPDATE” și „DELETE” [13].

Un sistem popular de gestionare a bazelor de date relaționale este Microsoft SQL Server, ce oferă o platformă robustă și scalabilă, pentru a gestiona și stoca eficient date. În acest mod, Microsoft SQL Server oferă unelte și funcționalități avansate, pentru a administra și dezvolta baze de date. Câteva exemple ar fi: suportul pentru tranzacții, securitate crescută, optimizare a interogărilor și a replicării datelor, etc [14].

De asemenea, o altă funcționalitate interesantă oferită de Microsoft SQL Server este integrarea ușoară cu ASP.NET Core. Așadar, acest sistem poate fi utilizat pentru gestionarea bazelor de date, în strânsă relație cu un server BE, iar dezvoltatorii software pot utiliza Microsoft SQL în efectuarea interogărilor, inserărilor, actualizărilor și ștergerilor de date. O asemenea legătură strânsă între ASP.NET Core și Microsoft SQL Server este asigurată de către ADO.NET, sau ActiveX Data Objects.NET. Aplicațiile dezvoltate în ASP.NET Core comunică cu serverul bazei de date Microsoft SQL Server printr-un set de biblioteci și componente oferite de ADO.NET. Acesta din urmă, permite manipularea datelor între aplicația Back-End și baza de date propriu-zisă, prin executarea de comenzi SQL [14].

Deoarece atât ASP.NET Core, cât și Microsoft SQL Server au fost dezvoltate de Microsoft, aplicațiile web ce utilizează aceste tehnologii beneficiază atât de o compatibilitate excelentă între cele două componente, cât și de performanță ridicată. Bazele de date ce utilizează soluția oferită de Microsoft au capacitatea de a gestiona cantități mai mari de date, prelucrând cereri fără a afecta cu mult performanța totală. Acest aspect devine unul esențial pentru aplicațiile dezvoltate în ASP.NET Core, întrucât ele trebuie să răspundă rapid și eficient nevoilor utilizatorilor.

Din moment ce numărul dezvoltatorilor care utilizează unelte Microsoft a crescut foarte mult, cei de la Microsoft nu au neglijat nici aspectul de securizare. Sistemul Microsoft SQL Server dispune de funcționalități avansate de securitate, garantând un nivel ridicat de securitate a datelor. Algoritmii de securizare includ criptarea datelor, posibilitatea de autentificare și autorizare, bazate pe roluri diferite, protecție împotriva atacurilor de tip SQL Injection și așa mai departe. De asemenea, tranzacțiile efectuate asupra bazelor de date în cadrul sistemelor Microsoft SQL Server sunt tranzacții de tip ACID („Atomicity, Consistency, Isolation, Durability”), sistemul oferind suport pentru indexări eficiente [14].

Nu în ultimul rând, un avantaj cheie al platformei de la Microsoft este suportul solid primit din partea firmei, dar și din partea comunității largi de dezvoltatori software. Acest lucru indică faptul că este o soluție apreciată și pune la dispoziție diverse surse de ajutor, precum documentație consistentă, ușor de urmărit și bine structurată, forumuri oficiale pentru a pune întrebări și a rezolva probleme, sau bloguri oficiale pentru pasionați.

3.2.3 HTML

HTML, cunoscut sub numele de HyperText Markup Language, este un limbaj de marcare, folosit pentru a defini modul în care conținutul este prezentat și structurat pe paginile web. Am putea spune că HTML funcționează ca un schelet al unei pagini web, oferind instrucțiuni despre cum trebuie formatat și organizat conținutul ce urmează a fi afișat de către browser-ul web. Astfel, acest limbaj reprezintă unul dintre pilonii fundamentali ai web-ului, oferind posibilitatea de a crea pagini web interactive și ușor accesibile [13].

O pagină HTML este compusă din secțiuni de componente, delimitate prin intermediul unui set de etichete (în engleză numite „tags”). La bază, denumirea fiecărei etichete este încadrată între simbolurile „<” și „>”. Spre exemplu, tag-ul „<h1>” desemnează un titlu de nivel 1, iar „<p>” marchează un paragraf de text. Fiecare astfel de etichetă conține două părți: semnele de început „<>” și semnele de încheiere „</>”. Un model sugestiv de

asemenea perechi poate fi eticheta următoare: `<div> ... </div>`, unde în locul punctelor se pot adăuga alte etichete dorite, acestea reprezentând o secțiune de document bine delimitată. Utilizarea tag-ului „div” este folosită pentru a defini secțiuni cu informații și stiluri diferite [13].

Cel mai important element al unei pagini HTML este tag-ul ce poartă aceeași denumire: „`<html>`”. Acesta este format din două secțiuni, una pentru capul paginii, cu eticheta „`<head>`”, iar a doua pentru corpul paginii, delimitată de „`<body>`”. Capul paginii este format din detalii despre pagina web, cum ar fi titlul, meta-datele, legăturile către stiluri CSS sau scripturi JS. Pe de altă parte, corpul paginii este secțiunea care are conținutul real al paginii, ce include text, imagini, link-uri și multe alte elemente. Deoarece în consistența unei pagini HTML se află un număr mare de componente, acest limbaj organizează elementele într-o structură ierarhică. Din acest motiv, aceste pagini au o structură mai complexă, prin înglobarea elementelor unul în altul [13].

În plus față de exemplele deja menționate în acest subcapitol, mai sunt disponibile o varietate de etichete, pentru a defini sau formata conținutul unei pagini. Alte exemple ar fi: eticheta `<img>` ce permite inserarea imaginilor, combinațiile de tag-uri `<ul>` și `<li>` pentru definirea listelor neordonate și a elementelor acestor liste, etichetele `<table>`, `<tr>` și `<td>` pentru a crea tabele împărțite pe rânduri și coloane sau tag-ul `<a>`, cu atributul „href” pentru conexiunile cu alte pagini, ce îi permit utilizatorului să se deplaseze în pagini sau secțiuni diferite în cadrul aceluiasi site web [13].

Atunci când vine vorba de relația dintre limbajele HTML și ASP.NET Core, HTML este utilizat pentru a defini și structura conținutul paginilor web ale aplicațiilor ce utilizează un server Back-End dezvoltat în ASP.NET. Acesta din urmă oferă suport integrat pentru crearea de pagini HTML dinamice, permițând programatorilor să dezvolte site-uri web interactive. În plus, conceptele de legare a datelor („Data Binding”) și modele de vizualizare („View Models”) facilitează interacțiunea dintre un server și partea de client HTML, deoarece „Data Binding” presupune legarea datelor din baza de date, în mod direct la elementele de interfață ale paginilor web (acest lucru înseamnă că dacă datele se schimbă în timp real, acestea sunt actualizate automat și în interfața vizuală a site-ului web), iar conceptul de „View Models” organizează și separă informațiile, servind ca o abstracțiune între datele salvate și interfața utilizatorului (acest mecanism permite dezvoltatorilor un sistem de gestionare ce nu amestecă logica aplicației cu logica de prezentare) [11].

3.2.4 CSS

În crearea paginilor web mai este folosit și limbajul CSS (pe lung, „Cascading Style Sheets”) pentru stilizare. Acest limbaj este folosit pentru a da viață paginilor web, adăugând o varietate de elemente vizuale și efecte estetice. Întrucât acest limbaj de stilizare se ocupă doar cu înfrumusețarea și așezarea conținutului pe pagină, el depinde în mod direct de HTML, acesta devenind un limbaj neutru din punct de vedere al stilizării conținutului. În loc să se definească direct în HTML stilurile și aspectul vizual al paginii, acesta doar importă acest tip de configurări din fișiere ce au extensia „.css” [13].

Dezvoltatorii pot modifica aspectul fiecărui element HTML individual, dar și al grupurilor de elemente, utilizând CSS. Acest lucru este posibil prin utilizarea selectorilor CSS pentru a alege elementele dorite și apoi aplicarea proprietăților și valorilor aferente. Spre exemplu, culorile de fundal, fonturile, decorațiunile de text, dimensiunile, marginile, alinierea și multe alte caracteristici vizuale pot fi modificate în CSS, cu ușurință [13].

Acest limbaj de stilizare are la bază conceptul de cascading, care se referă la prioritizarea și ierarhia regulilor de stil. Astfel, dacă sunt prezente două sau mai multe reguli contradictorii, aplicate asupra aceluiași element HTML, ordinea anterioară este utilizată în funcție de specificul selectorilor și ordinea în care aceștia sunt definiți în fișierul CSS. Cu ajutorul acestei cascade, programatorii pot controla în mod eficient aspectul unei pagini web, moștenind sau suprascriind stilurile componentelor HTML în funcție de nevoi [13].

Un alt concept important, utilizat în cadrul limbajului CSS, este conceptul de „box model”, ce are ca rol stabilirea modului în care elementele HTML sunt prezentate grafic. Am putea privi fiecare componentă HTML ca pe o „cutie” cu margini, borduri, padding și conținut. Dezvoltatorii software au puterea de a crea structuri complexe și flexibile, modificând dimensiunea și poziționarea elementelor, cu proprietățile puse la dispoziție de CSS. Totodată, pe lângă stilizarea paginilor web statice, acest limbaj poate fi folosit și pentru a adăuga animații, tranziții sau efecte unice. Aceste funcționalități sunt aduse de CSS3, cea mai recentă versiune, ce permite definirea transformărilor, opacităților, umbrelor, gradienturilor și a multor alte efecte fascinante [13].

3.2.5 JAVASCRIPT

JavaScript este un limbaj de programare interpretat, orientat pe obiecte și folosit adeseori pentru a oferi paginilor web funcționalități adiționale, interactive și dinamice. Acesta funcționează la fel ca HTML, în navigatorul web, oferind capacitatea de a manipula și controla comportamentul a diverselor elemente HTML, de a gestiona evenimente la intervale de timp sau declanșate de anumite acțiuni, de a comunica în mod direct cu serverul și multe alte operațiuni specifice comportamentului unei pagini web [13].

Astăzi, majoritatea browser-elor web moderne utilizează acest limbaj versatil și flexibil. Dezvoltatorii folosesc JavaScript pentru a face paginile web mai interactive, prin diverse efecte de animație, validări ale formularelor și actualizări dinamice ale conținutului afișat pe pagină. În plus, JavaScript permite comunicarea asincronă cu partea de Back-End, prin tehnologia AJAX („Asynchronous JavaScript and XML”). Această tehnologie include utilizarea de JavaScript și XML în mod asincron, pentru a actualiza paginile web fără reîncărcare completă [13].

O caracteristică esențială a JavaScript-ului este manipularea DOM (Document Object Model), ce oferă accesul și modificarea structurală a elementelor HTML. Cu ajutorul funcțiilor JavaScript, programatorii pot selecta, adăuga și șterge elemente, sau modifica proprietăți ale elementelor existente. De asemenea, acest limbaj susține tehnica de programare orientată pe obiecte (OOP). Datorită acestui lucru, se pot implementa clase, obiecte și metode pentru a structura și organiza mai bine codul, facilitând astfel întreținerea aplicațiilor de complexitate mare, sau reutilizarea bucăților de cod [13].

În al doilea rând, JavaScript mai beneficiază și de o multitudine de framework-uri și biblioteci, precum jQuery, React, Angular, Vue și multe altele. Acestea sunt foarte avantajoase, fiindcă extind funcționalitatea de bază a limbajului și îmbunătățesc mediul de dezvoltare. Pentru a construi aplicații web puternice și scalabile, aceste extensii simplifică interacțiunea cu DOM-ul, oferind metode mai simple și concise pentru a scrie un cod structurat.

Un avantaj major în interacțiunea JavaScript-ului cu ASP.NET Core este faptul că JS este integrat în mod nativ în ASP.NET Core, fapt ce facilitează comunicarea și schimbul de date între client și server. Această integrare oferă o experiență fluidă, iar dezvoltatorii pot crea mai ușor aplicații web de tip „full-stack”, utilizând JavaScript în combinație cu HTML și CSS pentru partea de client.

3.2.6 REACT JS

React reprezintă una dintre cele mai folosite biblioteci JavaScript, disponibilă open-source și este folosită pentru a crea interfețe de utilizator (prescurtate ca UI) dinamice și reutilizabile. Această bibliotecă a fost creată de Facebook și a ajuns, pe parcursul anilor, una dintre cele mai apreciate biblioteci JS de către comunitatea de dezvoltatori software. React utilizează un concept numit „Virtual DOM”, ce permite elementelor UI-ului să fie actualizate cu succes în timp real, fără a mai fi nevoie de a reîncărca întreaga pagină [15].

Principalul atribut al acestei biblioteci este accentul pus pe componentizare. Programatorii au posibilitatea de a implementa componente complet independente, reutilizabile, ce pot fi combinate mai apoi pentru a crea interfețe complexe. De asemenea, programatorii pot scrie un cod bine organizat, ușor de înțeles și întreținut, prin împărțirea interfeței în componente diferite [15].

Un alt concept important în React îl reprezintă fluxul de date unidirecțional. Folosind această bibliotecă, datele sunt gestionate într-un mod predictibil, iar interfața utilizatorului afișează automat modificările stării elementelor. Acest tip de abordare ajută la menținerea consistenței codului și la sincronizarea dintre diversele componente. Pe lângă fluxul de date unidirecțional, React mai utilizează și JSX (acronim de la JavaScript XML), ce permite definirea componentelor folosind o sintaxă asemănătoare cu XML, în scripturile JS. În plus, logica și structurile de control pot fi utilizate în timpul construirii componentelor, cu ajutorul JSX [15].

Ca și alte tehnologii menționate în acest capitol, React beneficiază, la rândul său, de o comunitate bogată și de o mulțime de framework-uri sau biblioteci special concepute pentru React. Câteva exemple ar fi „React Router”, pentru gestionarea rutărilor, „Redux” pentru gestionarea stării aplicației sau „Material-UI” pentru utilizarea rapidă și ușoară a componentelor de UI predefinite. Aceste pachete, dezvoltate special pentru biblioteca React, adaugă un set de funcționalități avansate, facilitând dezvoltarea clienților web cu această bibliotecă. Totodată, prin utilizarea DOM-ului virtual, performanța bibliotecii este crescută, minimizând numărul de modificări efectuate în DOM-ul real al site-ului web, rezultând într-o experiență îmbunătățită pentru utilizatori [15].

4. PROIECTAREA APLICAȚIEI

4.1 ANALIZA CERINȚELOR SISTEMULUI

În această secțiune se vor prezenta caracteristicile și comportamentul platformei UNIVENT, prin intermediul cerințelor acestui sistem. Cerințele unei aplicații software se împart în două categorii: cerințe funcționale și cerințe non-funcționale. Cerințele funcționale sunt menite să prezinte ce anume ar trebui să facă o aplicație software și cum să se comporte în anumite scenarii. Pe de altă parte, cerințele non-funcționale definesc calitatea și performanța sistemului, conform criteriilor sau standardelor respectate în dezvoltarea aplicației. Întrucât ambele tipuri de cerințe sunt esențiale pentru a crea aplicații cu succes, în cadrul acestui subcapitol vor fi descrise ambele tipuri de cerințe.

4.1.1 CERINȚE FUNCȚIONALE

Aplicația UNIVENT permite utilizarea a două roluri diferite: un administrator de sistem, prescurtat ca „admin” și utilizator simplu („user” în engleză), pentru studenți. Astfel, din perspectiva *administratorului*, aplicația permite:

- 1) **Autentificarea și deconectarea** din cont. Autentificarea este un proces esențial în care utilizatorul furnizează informațiile de identificare confidentiale, introduse la momentul înregistrării în sistem. Utilizatorul completează câmpurile obligatorii, cum ar fi numele de utilizator/adresa de e-mail și parola, în interfața aplicației. Aceste date sunt apoi verificate de către sistem pentru a confirma autenticitatea lor. După ce sistemul validează cu succes datele primite, utilizatorul primește permisiunea de a accesa platforma. Totodată, sistemul va observa rolul curent al utilizatorului, iar dacă acesta este administrator, va fi redirecționat pe pagina de setări exclusive rolului de administrator. De asemenea, sesiunea de autentificare este încheiată atunci când utilizatorul se deconectează din aplicație.
- 2) **Adăugarea unei universități noi** în lista de universități acceptate. Întrucât platforma UNIVENT va fi disponibilă studenților de la universitățile partenere, administratorul de sistem va putea insera o nouă universitate, care este de acord cu această aplicație și dorește să se alăture acestui proiect.
- 3) **Modificarea numelui unei universități existente**. În cazul în care o universitate parteneră dorește schimbarea numelui de pe platformă, administratorul poate edita acest câmp cu ușurință.
- 4) **Ștergerea unei universități existente**. Dacă o universitate parteneră dorește să renunțe la colaborare, administratorul poate șterge universitatea respectivă din listă, în câteva click-uri. Astfel, vor fi eliminate de pe platformă toate informațiile personale și recenziile studenților afectați, evenimentele create de aceștia și participările lor în cadrul evenimentelor create de studenții altor universități.

- 5) **Adăugarea unui tip nou de evenimente.** Aplicația UNIVENT pune la dispoziție o listă de tipuri de evenimente suportate pentru studenți, iar administratorul platformei poate adăuga un tip nou de eveniment, la cererea studenților și cu acordul universităților partenere.
- 6) **Modificarea numelui unui tip de evenimente existent.** Dacă cumva va fi necesară editarea unui tip de eveniment, administratorul poate realiza sarcină, în mod similar cu modificarea numelui unei universități.
- 7) **Ștergerea unui tip de evenimente existent.** În cazul în care una dintre universitățile partenere dorește să se elimine un anumit tip de evenimente, administratorul îl poate șterge, pierzându-se astfel toate evenimentele create cu acel tip de evenimente setat ca opțiune.
- 8) **Aprobarea conturilor noi de studenți.** Administratorul de sistem primește o listă de conturi noi de studenți ce au fost create, iar după confirmarea datelor cu universitatea în cauză, acesta poate să aprobe pe site un utilizator.

În continuare, este prezentă lista caracteristicilor platformei, pentru *utilizatorii simpli*:

- 1) **Crearea unui cont de student.** Sistemul permite înregistrarea gratuită în platformă pentru orice student nou, înrolat la una dintre universitățile partenere. Acesta va completa formularul cu datele personale (o fotografie de profil este opțională), după care se va crea contul în baza de date. După ce contul nou a fost aprobat, studentul va putea accesa contul prin autentificare.
- 2) **Autentificarea și deconectarea din cont.** Și de această dată, studentul completează câmpurile obligatorii, cum ar fi numele de utilizator/adresa de e-mail și parola, în interfața aplicației. Aceste date sunt apoi verificate de către sistem pentru a confirma autenticitatea lor. După ce sistemul validează cu succes datele primite, utilizatorul primește permisiunea de a accesa platforma. Aplicația Front-End va verifica rolul utilizatorului, primit în răspunsul de la Back-End, iar dacă acesta este un student, va fi redirecționat pe pagina aferentă studenților. De asemenea, sesiunea de autentificare este încheiată atunci când utilizatorul se deconectează din aplicație.
- 3) **Vizualizarea evenimente existente.** După ce se autentifică și este autorizat, studentul va putea vizualiza o listă cu evenimentele disponibile în acel moment. Pe pagina unde sunt prezentate evenimentele disponibile, există și opțiuni mai avansate precum filtrarea în funcție de tipul de eveniment și o bară de căutare, ce va afișa rezultatele obținute după numele evenimentelor.
- 4) **Vizualizarea detaliilor unui eveniment existent.** Orice student poate vizualiza detaliile unui eveniment creat și lista de participanți.
- 5) **Înrolarea ca participant la un eveniment.** Pe pagina cu detalii ale unui eveniment, orice student se poate înrola ca participant, în limita numărului maxim de participanți și a locurilor disponibile la acel moment. De asemenea, studentul se poate înrola ca participant doar dacă nu este deja participant și nu este cel care a creat evenimentul.

- 6) **Opțiunea de a crea un eveniment public.** Orice student autentificat în cont are dreptul de a crea un eveniment nou, completând detaliile și opțiunile specifice, în funcție de nevoie. Aceste date vor fi validate de către server, înainte de salvarea acestora în baza de date. Cu toate acestea, studentul își va asuma rolul de gazdă, sau îndrumător pentru restul participanților și se va asigura că totul va decurge bine, luând măsuri pentru prevenirea oricăror incidente nedorite.
- 7) **Vizualizarea informațiilor profilului personal.** Utilizatorul poate verifica informațiile sale personale, în orice moment. Tot pe această pagină, este prezent un rating, obținut ca medie aritmetică a tuturor recenziilor primite și un istoric al profilului. Acest istoric oferă detalii despre data creării contului, o listă cu toate evenimentelor create de acest student și o listă cu toate evenimentele în care studentul s-a înrolat, indiferent dacă evenimentul a avut loc sau e doar planificat.
- 8) **Opțiunea de a edita informațiile unui eveniment creat.** Studentul își poate vizualiza toate evenimentele create și poate alege să editeze orice eveniment creat de acesta, care încă nu a avut loc și care nu a fost anulat. Studentul gazdă este responsabil de actualizarea detaliilor evenimentului, în cazul schimbării de locație, ajustare a numărului de participanți sau alte informații.
- 9) **Opțiunea de a anula un eveniment creat.** Pe pagina unde sunt toate evenimentele create de utilizatorul autentificat în platformă, există și opțiunea de a anula un eveniment. Studentul care a creat evenimentul are dreptul de a-l anula, oricând înaintea începerii.
- 10) **Opțiunea de a părăsi un eveniment, ca participant.** Utilizatorii care s-au înrolat ca participanți la evenimente create de alte persoane, pot părăsi evenimentul dacă cumva s-au răzgândit sau a intervenit ceva.
- 11) **Editarea informațiilor personale din profil.** În situația în care datele personale ale unui student nu mai coincid cu cele reale sau studentul alege să își schimbe adresa de e-mail, platforma UNIVENT permite editarea acestora în siguranță.
- 12) **Schimbarea parolei.** Studenții care doresc schimbarea de parolă, au această opțiune prezentă. Aceștia trebuie doar să introducă parola veche și pe cea nouă. Parola veche este verificată, pentru a crește securitatea platformei, iar dacă serverul aprobă această cerere, parola studentului va fi înlocuită cu cea nouă. Tot din motive de securitate, odată schimbată parola, studentul va fi deconectat din cont, pe loc, fiind necesară reconectarea cu parola nouă.
- 13) **Opțiunea de a acorda recenzii altor utilizatori.** Pentru a stabili un mediu online prietenos și primitor, UNIVENT folosește funcționalitatea de rating pentru fiecare utilizator. Odată încheiat un eveniment, orice participant poate să acceseze oricând dorește un formular de recenzii, pentru restul participanților și studentul care a creat evenimentul. Cu toate acestea, completarea feedback-ului este complet opțională, însă formularul se poate trimite o singură dată per eveniment.
- 14) **Opțiunea de a șterge contul.** În cazul în care un student alege să părăsească în mod definitiv platforma UNIVENT, acesta poate opta pentru ștergerea

permanentă a contului. Această acțiune va șterge tot istoricul utilizatorului pe platformă și datele personale salvate.

4.1.2 CERINȚE NON-FUNCȚIONALE

Pentru a oferi o experiență de utilizare cât mai bună, trebuie luate în considerare un set de cerințe non-funcționale în dezvoltarea aplicațiilor folosind tehnologiile ASP.NET Core și React. În cadrul acestei secțiuni vor fi prezentate, sumar, câteva cerințe non-funcționale respectate de platforma UNIVENT.

Utilizatorii doresc întotdeauna să se bucure de o aplicație care funcționează rapid și într-un mod receptiv la comenzile date, astfel o primă cerință non-funcțională este reprezentată de **performanța** oferită. Performanța crescută are ca rezultat încărcarea rapidă a paginilor și comenzi ale utilizatorilor executate rapid. Se poate asigura o performanță ridicată utilizând tehnici pentru optimizarea numărului de cereri și răspunsuri trimise între client și server, îmbunătățirea algoritmilor de căutare în baza de date, utilizarea eficientă a resurselor de rețea și server, combinarea sau minimizarea fișierelor ce definesc dimensiunea paginii web, stocare locală a anumitor resurse des accesate și multe altele.

O altă cerință care definește calitatea unei pagini web este **scalabilitatea**, deoarece platforma web trebuie să poată gestiona un număr mai mare de utilizatori, ceea ce implică un trafic de date crescut. Din punct de vedere al scalabilității, aplicația trebuie să dețină abilitatea de a gestiona cu succes scenariile de creștere a volumului de date sau utilizatori, fără a afecta performanța sau disponibilitatea resurselor.

O problemă destul de mare întâlnită în dezvoltarea și implementarea aplicațiilor web este gestionarea datelor sensibile ale utilizatorilor. Astfel, o a treia cerință non-funcțională, dar esențială, este **securitatea** datelor. Autentificarea și autorizarea utilizatorilor pe platformă, protecția parolelor prin criptare sau implementarea mecanismelor de protecție împotriva accesării de către surse ne-autorizate sunt cele mai bune exemple pentru practici de securitate.

Nu în ultimul rând, o altă cerință non-funcțională vizată de programatori în dezvoltarea site-urilor web moderne este **gradul de utilizabilitate**. Acest aspect crucial în facilitarea unei experiențe plăcute pentru utilizatori impune ca aplicația să fie accesibilă și cât mai ușoară de utilizat, pentru toate categoriile de utilizatori. În cazul platformei UNIVENT, utilizatorii principali sunt studenți, tineri ce au vârste apropiate, însă acest scenariu este doar un caz izolat, în comparație cu restul site-urilor web. Orice aplicație web trebuie să ofere o interfață grafică cât mai prietenoasă, ușor de înțeles, intuitivă pentru a găsi toate funcționalitățile. De asemenea, pentru a evita scenariile neplăcute, o bună practică este utilizarea ferestrelor de dialog pentru confirmarea anumitor acțiuni ale utilizatorilor, ce pot fi ireversibile în unele cazuri. Totodată, limbajul utilizat pe pagină este familiar tuturor, fiind simplu și clar. Prin respectarea principiilor enunțate în acest paragraf, se poate asigura un produs ce oferă o experiență de utilizare plăcută și eficientă, în care utilizatorii pot interacționa cu aplicația ușor, fără a întâmpina dificultăți.

4.2 CAZURI DE UTILIZARE

Acest subcapitol va furniza o analiză detaliată a cazurilor de utilizare ale sistemului, în scopul de a înțelege mai bine funcționalitățile disponibile pentru fiecare tip de utilizator. Prin explorarea acestor scenarii, se dorește crearea unei perspective mai profunde asupra modului în care platforma poate fi utilizată cât mai eficient, dar și înțelegerea beneficiilor pe care le aduce utilizatorilor.

4.2.1 ACTORII SISTEMULUI

În cazul aplicației UNIVENT, actorii sistemului sunt în număr de doi, pentru rolurile de utilizatori disponibile: *administrator de sistem* sau *utilizator simplu*.

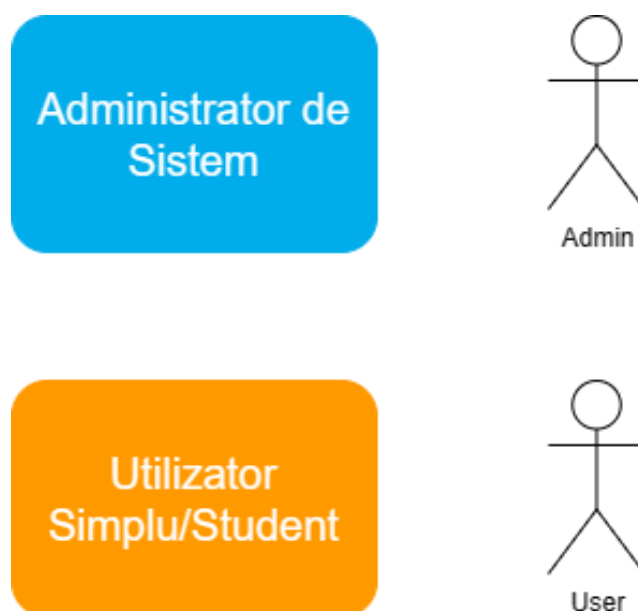


Figura 4.1: Actorii sistemului

Pentru a distinge mai ușor cele două categorii de utilizatori, figura de mai sus conține culori diferite, iar pentru simplitate, denumirile actorilor au fost prescurtate la „Admin” și „User”.

O diagramă „Use Case” este o reprezentare grafică a interacțiunilor dintre actori și sistemul software, într-un anumit context. Aceasta oferă o imagine de ansamblu a modului în care utilizatorii folosesc sistemul pentru a îndeplini anumite obiective. Diagrama Use Case identifică acțiunile specifice pe care utilizatorii le pot efectua în cadrul aplicației web, punând accentul pe relațiile dintre acțiuni și rezultatele așteptate. Acest lucru ajută părțile interesate, cum ar fi dezvoltatorii, proiectanții și clientul final, să comunice și să înțeleagă cerințele sistemului.

În continuare, vor fi prezentate diagramele de tip „Use Case”, pentru actorii aplicației UNIVENT.

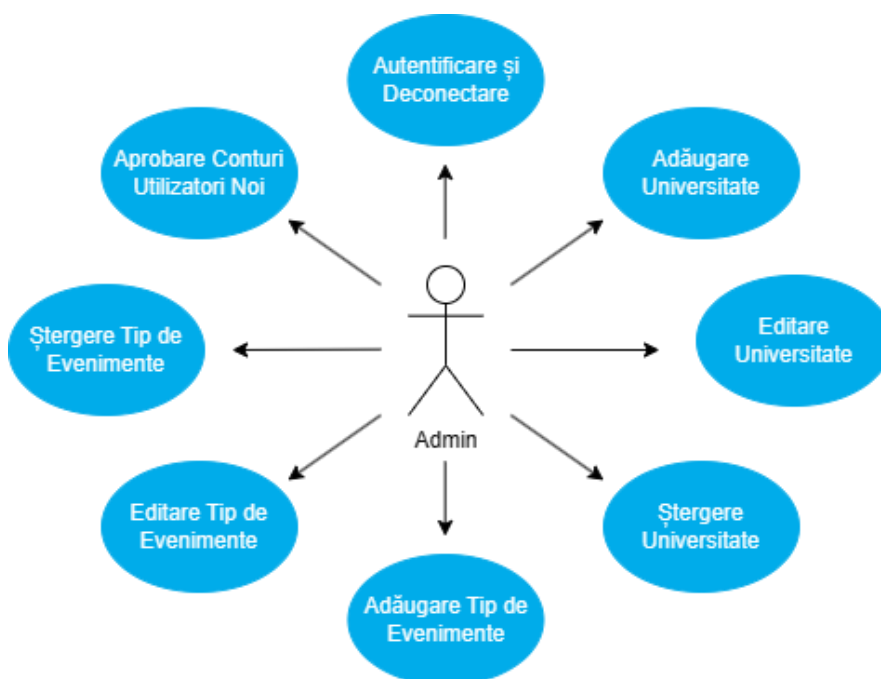


Figura 4.2: Cazuri de utilizare pentru administrator

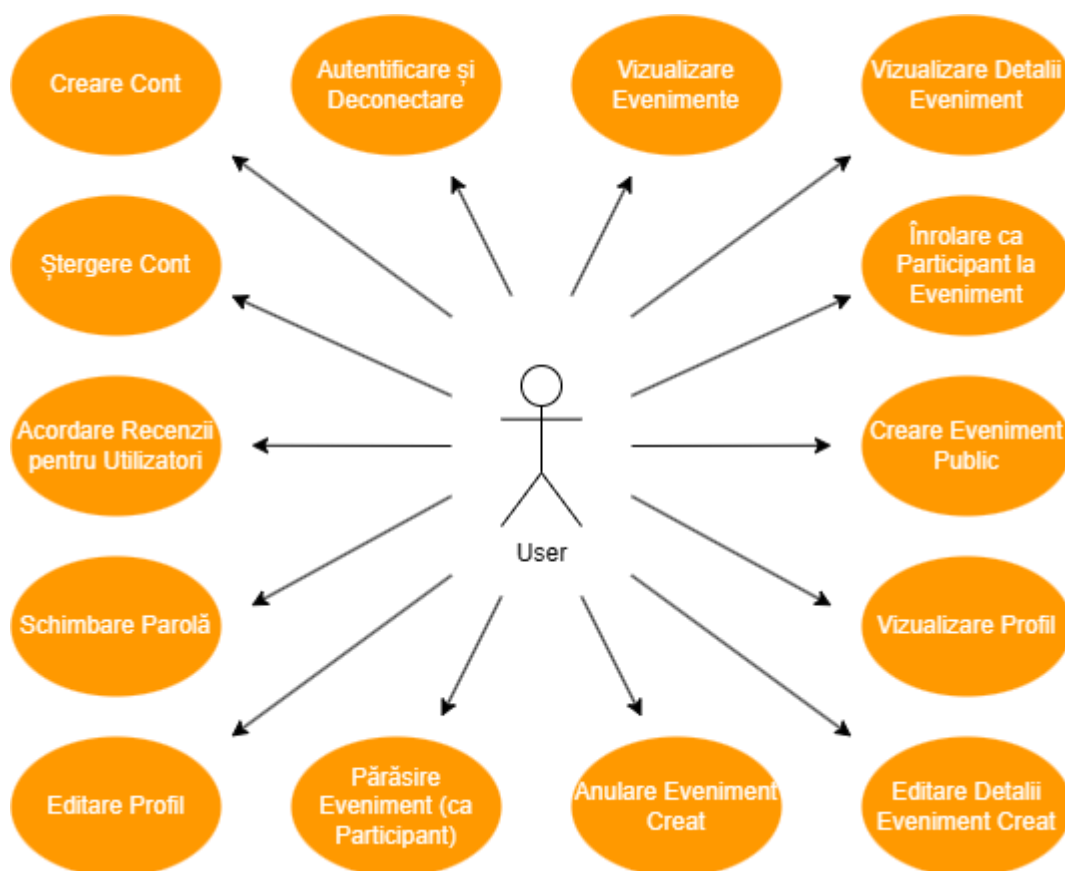


Figura 4.3: Cazuri de utilizare pentru student

Într-un sistem software, o diagramă de activitate este o reprezentare grafică a fluxului de activități și acțiuni care au loc în timpul unui proces sau funcționalități specifice. Aceasta ilustrează modul în care obiectele (entități, module sau alte componente) interacționează între ele și îndeplinesc diferite acțiuni într-un anumit context. Pentru a înțelege mai bine logica și comportamentul sistemului, mai jos se regăsesc diagramele de activitate pentru actorii platformei UNIVENT.

În primul rând, procesul de autentificare în cont și autorizare din partea serverului, este comun pentru ambele categorii de utilizatori. Astfel, în figura de mai jos, este prezentată logica și fluxul de acțiuni pentru partea de *autentificare și autorizare*.

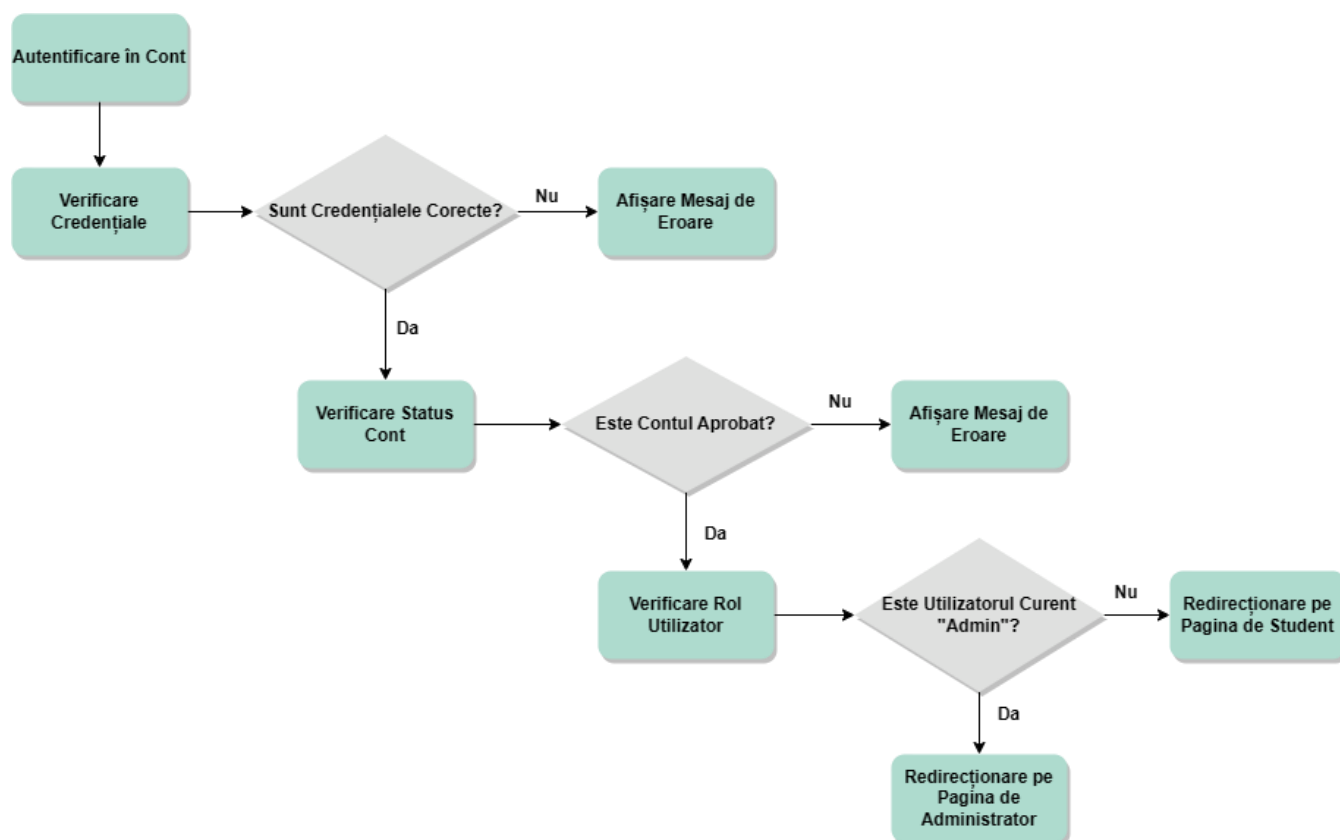


Figura 4.4: Diagrama de activitate a procesului de Autentificare și Autorizare

În continuare, este prezentată diagrama de activitate pentru rolul de *administrator* de sistem. Acest rol are responsabilitatea de a gestiona constant lista universităților partenere, în aplicație, tipurile de evenimente acceptate și procesul de aprobare a studenților noi.

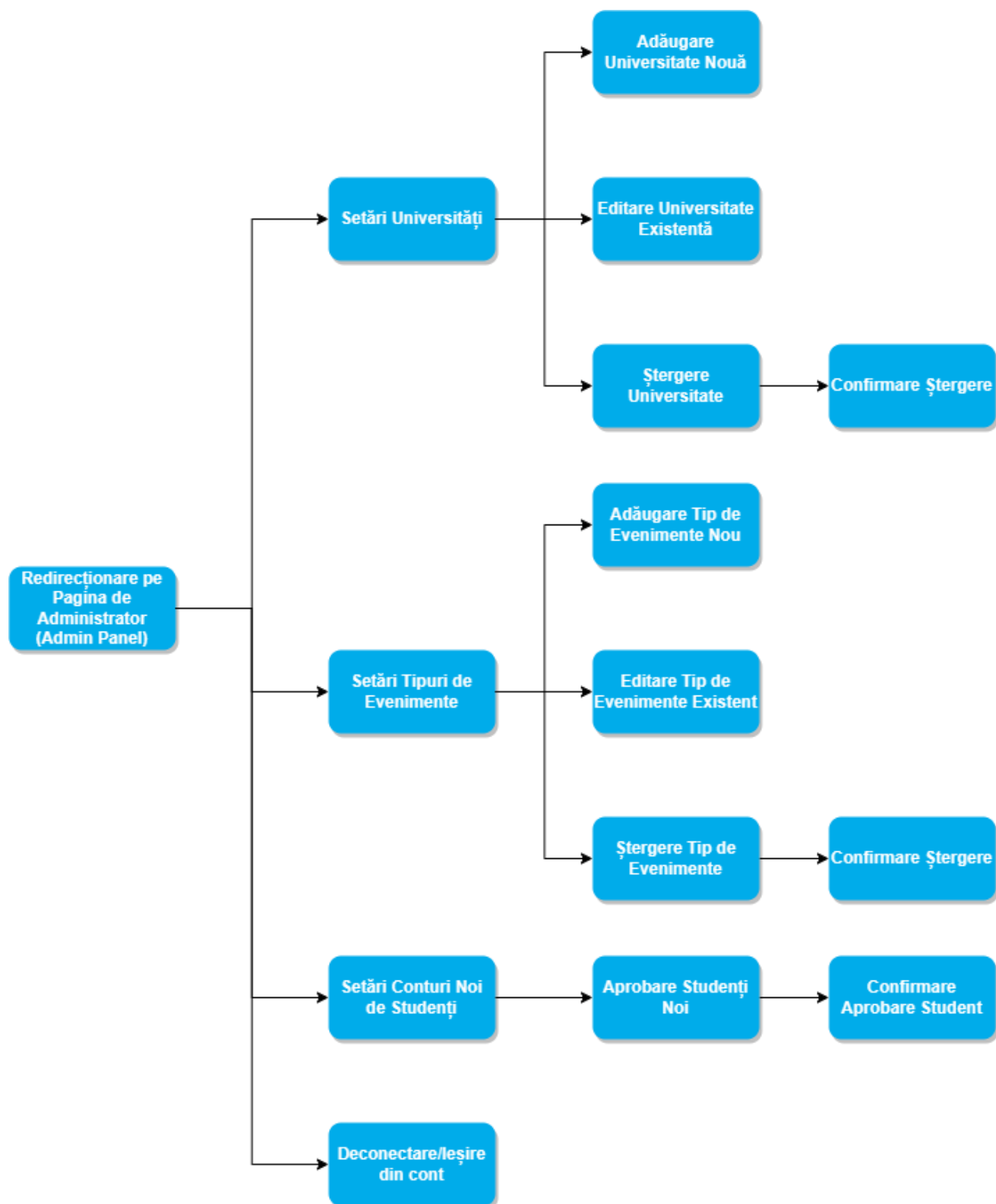


Figura 4.5: Diagrama de activitate pentru Administrator

În figura următoare, este prezentată diagrama de activitate pentru rolul de *utilizator simplu/student*.

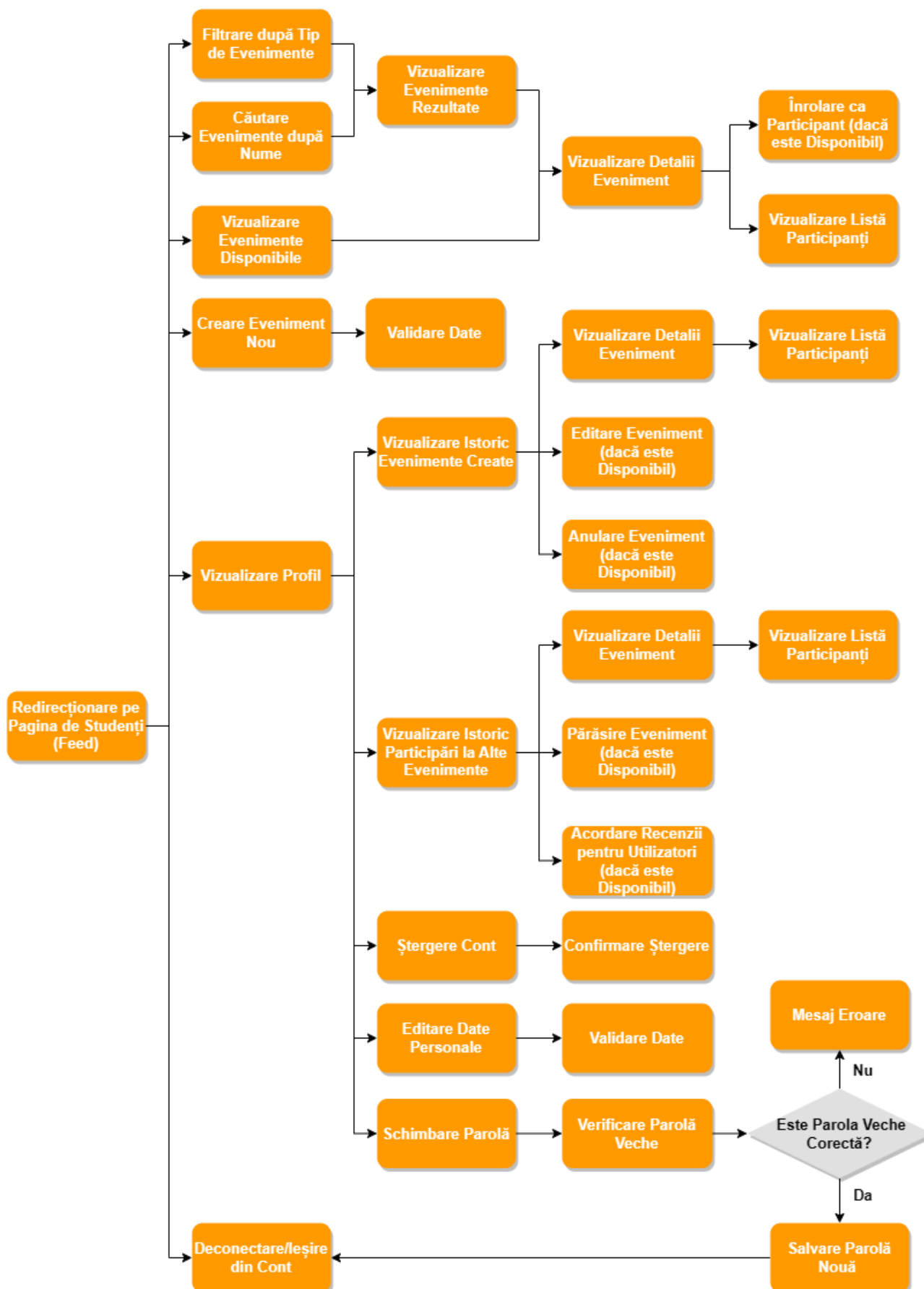


Figura 4.6: Diagrama de activitate pentru Student

5. IMPLEMENTAREA APLICAȚIEI

5.1 ARHITECTURA SISTEMULUI

Arhitectura utilizată în cadrul acestei aplicații este cea de tip client-server, în care părțile de Front-End și Back-End comunică între ele prin cereri și răspunsuri, iar cel din urmă are acces la baza de date, gestionând datele salvate. În acest mod, se permite conectarea individuală a mai multor clienți, la același server.

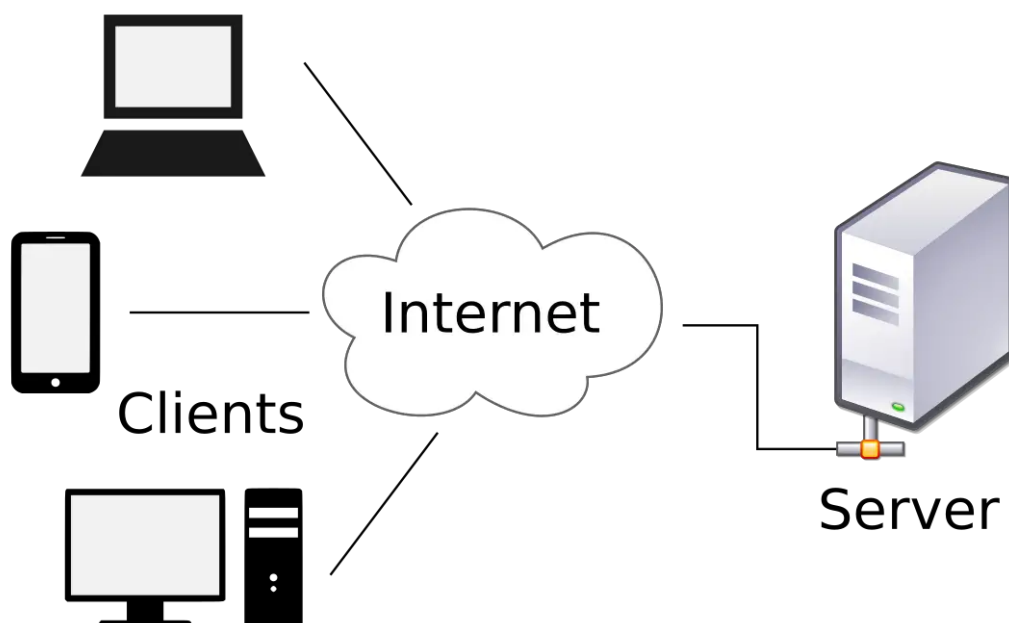


Figura 5.1: Arhitectura Client-Server [16]

5.2 PARTEA DE SERVER

Pentru componenta de Back-End, a fost implementat un API, ce respectă principiile REST, utilizând framework-ul ASP.NET Core, versiunea 6. Arhitectura principală folosită în cadrul acestei componente se numește „Clean Architecture” și este un concept popular în rândul dezvoltatorilor software. În cartea sa, Robert C. Martin (cunoscut sub porecla de „Uncle Bob”) pune accent pe faptul că fiecare componentă a unui sistem software are propriile sale responsabilități și funcționează în mod independent. În general, această arhitectură susține o proiectare modulară, care se concentrează pe logica de business, nu pe structura sau tehnologii specifice. Arhitectura Clean este compusă din mai multe niveluri, începând cu stratul intern, numit nivelul de Entități. Acesta conține regulile de business și informații despre entitățile sistemului software, care sunt încapsulate. Mai apoi, este prezent nivelul ce definește cazurile de utilizare specifice aplicației. Următorul strat este cel de Adaptoare de Interfață, fiind un intermediar între nivelul de Cazuri de Utilizare și exterior.

Stratul final, numit Cadre și Drive, prezintă detaliile tehnice, împreună cu tehnologiile utilizate pentru livrarea aplicației finale. Scopul arhitecturii Clean este de a crea un sistem complet independent de posibile probleme externe, dar ușor de înțeles pentru a fi întreținut în timp. Acest lucru le permite dezvoltatorilor să adapteze și să modifice sistemul software în funcție de cerințe noi [17]. În dezvoltarea platformei UNIVENT, s-au folosit patru niveluri principale, pentru partea de server: „Domain”, „Application”, „Api” și „Dal” (acronim de la „Data Access Layer”).

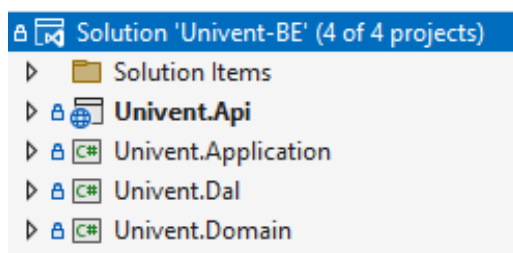


Figura 5.2: Structura proiectului Back-End în cadrul platformei UNIVENT

De asemenea, s-a ținut cont și de modelul MVC („Model-View-Controller”), ce împarte responsabilitățile sistemului în trei componente distincte: „Model”, „View” și „Controller”. Astfel, componenta model conține datele și logica de afaceri a aplicației, care este responsabilă de gestionarea și modificarea datelor. Vizualizarea este secțiunea din interfața grafică a aplicației, care afișează datele și permite interacțiunea cu acestea. Controllerul gestionează cererile utilizatorului și organizează interacțiunea dintre Model și Vizualizare. În cadrul modelului MVC, responsabilitățile sunt împărțite în mod clar, ceea ce duce la scrierea unui cod modular, mai extins și mai ușor de întreținut [11].

Stratul de Entități, numit Domain, conține modele ale tuturor entităților din baza de date, împreună cu informații specifice acestora. Mai mult, acest nivel are la bază conceptul de „Aggregates”, pentru a separa entitățile în funcție de tipul și rolul acestora.

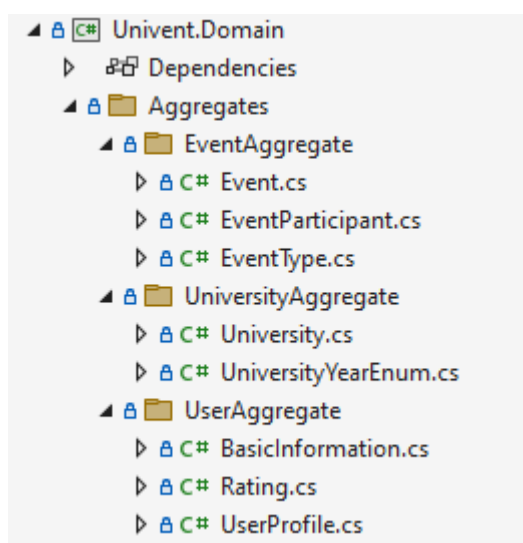


Figura 5.3: Structura nivelului Domain

Clasele prezente în acest nivel conțin câmpurile entităților, iar pentru inițializarea obiectelor, s-a folosit principiul de proiectare numit „Factory Pattern”. Acest principiu este utilizat deseori în programarea orientată pe obiecte, pentru a crea obiectele într-o manieră flexibilă, dar abstractizată. „Factory Pattern” separă procesul de creare al obiectelor, de utilizarea propriu-zisă a acestora, oferind o interfață comună, ce permite crearea unui număr mare de obiecte, fără a expune logica de creare [18]. Datorită utilizării acestui principiu arhitectural, constructorul fiecărei clase din stratul Domain a fost declarat privat.

```

1      using Univent.Domain.Aggregates.UserAggregate;
2
3      namespace Univent.Domain.Aggregates.UniversityAggregate
4      {
5          19 references
6          public class University
7          {
8              4 references
9              public Guid UniversityID { get; private set; }
10             private readonly List<UserProfile> _students = new List<UserProfile>();
11             1 reference
12             public IEnumerable<UserProfile> Students { get { return _students; } }
13             3 references
14             public string Name { get; private set; }
15
16             //Constructor
17             1 reference
18             private University()
19             {
20             }
21
22             //Factory method
23             1 reference
24             public static University CreateUniversity(string name)
25             {
26                 var newUniversity = new University
27                 {
28                     Name = name
29                 };
30
31                 return newUniversity;
32             }
33
34             //Public methods start here
35             1 reference
36             public void UpdateUniversity(string newName)
37             {
38                 Name = newName;
39             }
40         }
41     }

```

Figura 5.4: Clasă de Entitate din nivelul Domain

Nivelul Application reprezintă stratul de Cazuri de Utilizare și definește toate cazurile de utilizare ale sistemului, prin aplicarea logicii definite de dezvoltatori și coordonarea interacțiunii dintre entități. În implementarea acestuia, au fost folosite pachete adiționale, precum „MediatR”. Acest pachet oferă un obiect intermediar, ce transmite informațiile

cererilor din controller (prezent în nivelul Api), la stratul de Application, pentru a fi gestionate corespunzător.

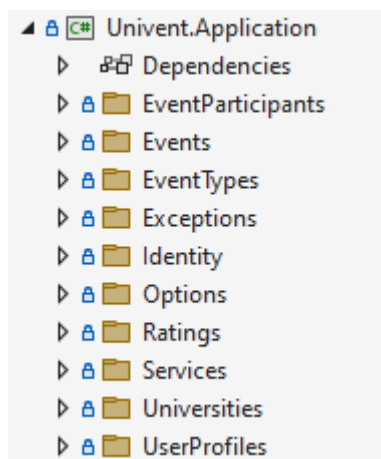


Figura 5.5: Structura nivelului Application

De asemenea, stratul Application conține un dosar pentru fiecare tip de entitate, în care au fost separate modelele cererilor de metodele care le gestionează. Modelele cererilor sunt împărțite în două categorii, în funcție de tip: „Commands” și „Queries”, fiecare tip având câte o metodă de gestionare, dedicată. În acest fel, s-a obținut un total de 4 dosare, respectând o abordare de tip CQRS („Command Query Responsibility Segregation”). Acest concept arhitectural separă operațiunile de comandă („Commands”) și operațiunile de interogare („Queries”), încurajând împărțirea logicii de scriere și a logicii de citire, în două părți distincte. Acest lucru le oferă programatorilor șansa de a optimiza, în mod corespunzător, fiecare componentă [19]. În cazul aplicației UNIVENT, operațiunile de comandă sunt dictate de solicitări de modificare a stării datelor, precum crearea, actualizarea și ștergerea datelor, iar operațiunile de interogare sunt utilizate pentru a colecta date, fără a afecta starea acestora. În acest mod, abordarea CQRS facilitează scalabilitatea și creșterea performanței, fiind o metodă flexibilă și eficientă de a crea aplicații complexe.

În serverul aplicației UNIVENT, clasele de tip Command și clasele de tip Query moștenesc interfața IRequest, iar clasele de tip Handler, pentru gestionarea operațiunilor și interogărilor, moștenesc interfața IRequestHandler. Aceste interfețe sunt disponibile în pachetul „MediatR”.

Un exemplu ilustrativ pentru abordarea de tip CQRS este reprezentat în figura de mai jos, care demonstrează structura de dosare pentru entitatea de tip Universitate. De menționat, este faptul că această abordare a fost utilizată pentru definirea logicii și comportamentului fiecărui tip de entitate din acest proiect. Totodată, au fost implementate toate scenariile și interacțiunile dintre entități, în manieră CQRS.

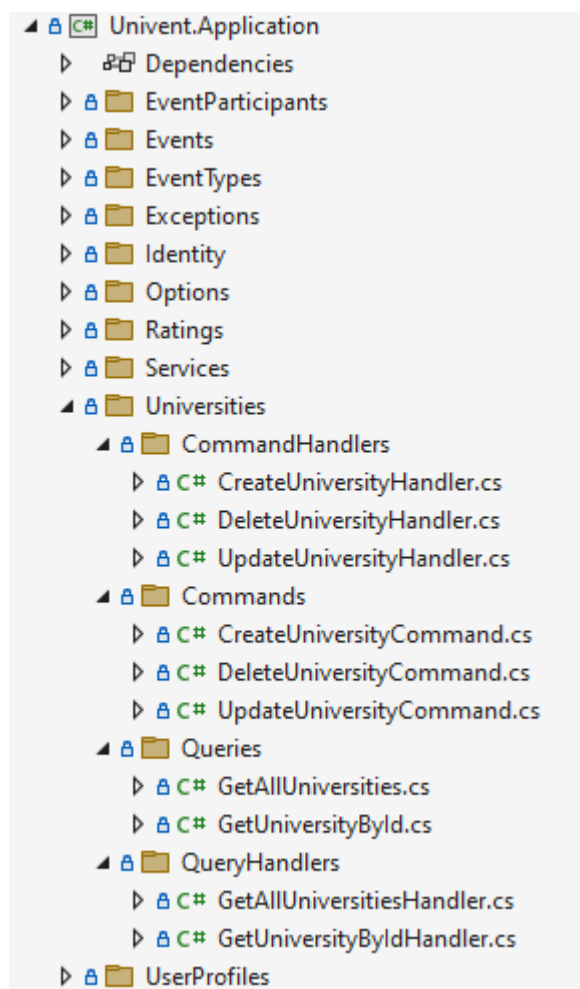


Figura 5.6: Abordare CQRS pentru entitatea Universitate

În figura de mai jos, este un model local pentru interogare, a entității de tip Universitate.

```
1  using MediatR;
2  using Univent.Domain.Aggregates.UniversityAggregate;
3
4  namespace Univent.Application.Universities.Queries
5  {
6      3 references
6      public class GetUniversityById : IRequest<University>
7      {
8          3 references
8          public Guid UniversityID { get; set; }
9      }
10 }
```

Figura 5.7: Model de Interogare pentru entitatea Universitate

În figura de mai jos, este prezentată o metodă de gestionare a cererii de tip Interogare, pentru entitatea de tip Universitate.

```

1  using MediatR;
2  using Microsoft.EntityFrameworkCore;
3  using Univent.Application.Exceptions;
4  using Univent.Application.Universities.Queries;
5  using Univent.Dal;
6  using Univent.Domain.Aggregates.UniversityAggregate;
7
8  namespace Univent.Application.Universities.QueryHandlers
9  {
10     1 reference
11     internal class GetUniversityByIdHandler : IRequestHandler<GetUniversityById, University>
12     {
13         private readonly DataContext _dbcontext;
14
15         0 references
16         public GetUniversityByIdHandler(DataContext dbcontext)
17         {
18             _dbcontext = dbcontext;
19
20         0 references
21         public async Task<University> Handle(GetUniversityById request, CancellationToken cancellationToken)
22         {
23             var university = await _dbcontext.Universities
24                 .FirstOrDefaultAsync(predicate: u => u.UniversityID == request.UniversityID, cancellationToken)
25                 ?? throw new ObjectNotFoundException(type: nameof(University), id: request.UniversityID);
26
27             return university;
28         }
29     }
30 }

```

Figura 5.8: Metodă de Gestionare a unei interogări, pentru entitatea Universitate

În continuare, mai jos este un exemplu de model local pentru o comandă, a entității de tip Universitate.

```

1  using MediatR;
2  using Univent.Domain.Aggregates.UniversityAggregate;
3
4  namespace Univent.Application.Universities.Commands
5  {
6     4 references
7     public class CreateUniversityCommand : IRequest<University>
8     {
9         1 reference
10        public string Name { get; set; }
11    }
12 }

```

Figura 5.9: Model de Comandă pentru entitatea Universitate

```
1 using MediatR;
2 using Univent.Application.Universities.Commands;
3 using Univent.Dal;
4 using Univent.Domain.Aggregates.UniversityAggregate;
5
6 namespace Univent.Application.Universities.CommandHandlers
7 {
8     1 reference
9     internal class CreateUniversityHandler : IRequestHandler<CreateUniversityCommand, University>
10     {
11         private readonly DataContext _dbcontext;
12
13         0 references
14         public CreateUniversityHandler(DataContext dbcontext)
15         {
16             _dbcontext = dbcontext;
17         }
18
19         0 references
20         public async Task<University> Handle(CreateUniversityCommand request, CancellationToken cancellationToken)
21         {
22             var university = University.CreateUniversity(name: request.Name);
23             _dbcontext.Universities.Add(entity: university);
24             await _dbcontext.SaveChangesAsync(cancellationToken);
25             return university;
26         }
27     }
```

Figura 5.10: Metodă de Gestionare a unei comenzi, pentru entitatea Universitate

În plus, stratul Application este responsabil și pentru partea de securizare, întrucât conține o metodă pentru generarea unei chei unice numite JWT („JSON Web Token”). JWT este un standard deschis pentru transmiterea securizată a informațiilor personale în format JSON, între mai multe părți. De cele mai multe ori, acest standard este utilizat pentru autentificarea și autorizarea utilizatorilor în aplicații web sau alte servicii. O cheie JWT este formată din trei părți principale: header, payload și o semnătură. Header-ul conține detalii despre algoritmul de criptare și tipul de cheie, payload-ul conține datele ce trebuie transmise în mod securizat (precum informații ale utilizatorului și drepturile de acces), iar semnătura este rezultatul criptării combinate ale header-ului, payload-ului și a unei chei secrete (utilizată în verificarea autenticității cheii JWT) [20].

În cazul platformei UNIVENT, algoritmul de criptare este „HmacSha256”, iar cheia a fost configurată să conțină:

- Un subiect, care reprezintă adeseori identificatorul unic sau adresa de e-mail a utilizatorului,
- Momentul expirării, în acest caz fiind la două ore de la crearea cheii,
- Audiența pentru care a fost creată cheia,
- Entitatea emitentă, cine a creat cheia,
- Informații adiționale.

De asemenea, metodele de gestionare a cererilor de creare cont și autentificare în cont, apelează serviciul de creare cheie JWT, adăugând informații adiționale, precum adresa de e-mail a utilizatorului, identificatorul unic al profilului de utilizator, sau rolul acestuia.

```

8 namespace Univent.Application.Services
9 {
10     public class IdentityService
11     {
12         private readonly JwtSettings _jwtSettings;
13         private readonly byte[] _key;
14
15         public IdentityService(IOption<JwtSettings> jwtOptions)
16         {
17             _jwtSettings = jwtOptions.Value;
18             _key = Encoding.ASCII.GetBytes(_jwtSettings.SigningKey);
19         }
20
21         private SecurityTokenDescriptor GetTokenDescriptor(ClaimsIdentity claimsIdentity)
22         {
23             return new SecurityTokenDescriptor()
24             {
25                 Subject = claimsIdentity,
26                 Expires = DateTime.Now.AddHours(value: 2),
27                 Audience = _jwtSettings.Audiences[0],
28                 Issuer = _jwtSettings.Issuer,
29                 SigningCredentials = new SigningCredentials(key: new SymmetricSecurityKey(key: _key),
30                                                             algorithm: SecurityAlgorithms.HmacSha256Signature)
31             };
32         }
33
34         public JwtSecurityTokenHandler TokenHandler = new JwtSecurityTokenHandler();
35
36         public SecurityToken CreateSecurityToken(ClaimsIdentity claimsIdentity)
37         {
38             var tokenDescriptor = GetTokenDescriptor(claimsIdentity);
39             return TokenHandler.CreateToken(tokenDescriptor);
40         }
41
42         public string WriteToken(SecurityToken token)
43         {
44             return TokenHandler.WriteToken(token);
45         }
46     }
47 }

```

Figura 5.11: Serviciul de creare al cheii JWT

```

47 var claimsIdentity = new ClaimsIdentity(claims: new Claim[]
48 {
49     new Claim(type: JwtRegisteredClaimNames.Sub, value: identityUser.Email),
50     new Claim(type: JwtRegisteredClaimNames.Jti, value: Guid.NewGuid().ToString()),
51     new Claim(type: JwtRegisteredClaimNames.Email, value: identityUser.Email),
52     new Claim(type: "IdentityId", value: identityUser.Id),
53     new Claim(type: "UserProfileId", value: userProfile.UserProfileID.ToString()),
54     new Claim(type: "Role", value: role.FirstOrDefault())
55 });
56
57 var token = _identityService.CreateSecurityToken(claimsIdentity);
58 return _identityService.WriteToken(token);

```

Figura 5.12: Crearea și adăugarea de informații adiționale în cheia JWT

Nu în ultimul rând, acest nivel al serverului este responsabil și pentru implementarea excepțiilor aplicației. Aceste excepții conțin mesaje personalizate, iar tratarea acestor excepții se realizează în următorul strat.

```
1 namespace Univent.Application.Exceptions
2 {
3     public class ObjectNotFoundException : ApplicationException
4     {
5         public ObjectNotFoundException(string type, object id)
6             : base(message: $"Object of type {type} with id {id} doesn't exist.")
7         {
8         }
9     }
10
11     public ObjectNotFoundException(string type, object id1, object id2)
12         : base(message: $"Object of type {type} with the combination of id1 {id1} and id2 {id2} doesn't exist.")
13     {
14     }
15 }
16
17 }
```

Figura 5.13: Exemplu de excepție personalizată

Un alt strat al serverului platformei UNIVENT este Api, ce poate fi corelat cu nivelul de Adaptoare de Interfață al arhitecturii Clean, întrucât dosarul principal, „Controllers”, are rolul de a primi cererile de la client și de a le trimite stratului de Application pentru a le executa și returna un răspuns aferent.

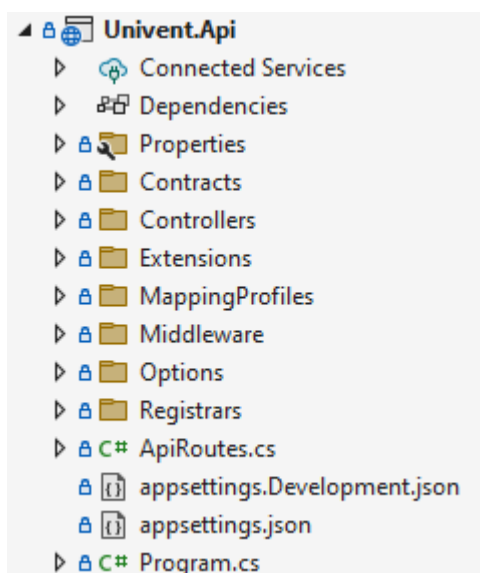


Figura 5.14: Structura nivelului API

În acest nivel, sunt prezente contractele, ce definesc atât informațiile pe care trebuie să le conțină cererile primite de la clientul web, cât și datele ce vor fi returnate acestuia, prin răspunsuri. De asemenea, informațiile din contractele cererilor au fost decorate cu anotări pentru a defini cerințe suplimentare, precum număr minim sau maxim de caractere, format de e-mail și altele.

```

1  using System.ComponentModel.DataAnnotations;
2
3  namespace Univent.Api.Contracts.University.Requests
4  {
5      2 references
6      public record UniversityCreate
7      {
8          [Required]
9          [StringLength(maximumLength: 50, MinimumLength = 3)]
10         0 references
11         public string Name { get; set; }
12     }
13 }

```

Figura 5.15: Contract pentru cerere de creare a unei entități de tipul Universitate

```

1  namespace Univent.Api.Contracts.University.Responses
2  {
3      4 references
4      public record UniversityResponse
5      {
6          0 references
7          public Guid UniversityID { get; set; }
8          0 references
9          public string Name { get; set; }
10     }
11 }

```

Figura 5.16: Contract pentru răspunsul unei entități de tipul Universitate

De asemenea, în nivelul Api se regăsesc și mapările acestor contracte la modele locale din stratul Application. Aceste mapări au fost realizate cu ajutorul pachetului „AutoMapper”, pentru a oferi nivelului Application o imagine despre cum arată cererile din partea clientului. Modelele locale din acest nivel ajută interfețele de tip Handler să gestioneze logica aplicației, aplicând acțiunile necesare.

```

1  using AutoMapper;
2  using Univent.Api.Contracts.Event.Requests;
3  using Univent.Api.Contracts.Event.Responses;
4  using Univent.Application.Events.Commands;
5  using Univent.Domain.Aggregates.EventAggregate;
6
7  namespace Univent.Api.MappingProfiles
8  {
9      1 reference
10     public class EventMappings : Profile
11     {
12         0 references
13         public EventMappings()
14         {
15             CreateMap<EventCreate, CreateEventCommand>();
16             CreateMap<EventUpdate, UpdateEventCommand>();
17             CreateMap<EventUpdate_CancelOption, UpdateEvent_CancelOptionCommand>();
18             CreateMap<Event, EventResponse>();
19         }
20     }
21 }

```

Figura 5.17: Exemplu de mapare a contractelor, pentru entitatea Eveniment

În plus, în nivelul Api este aplicat și conceptul de „Middleware”, pentru a gestiona excepțiile într-o manieră centralizată. Aceste excepții sunt aruncate în nivelul serverului, unde este definită logica aplicației, însă sunt tratate în stratul Api.

```
51 | | | | | case EventUpdateNotPossibleException:  
52 | | | | | case EventDeleteNotPossibleException:  
53 | | | | | case UnapprovedUserException:  
54 | | | | |     statusCode = StatusCodes.Status403Forbidden;  
55 | | | | |     break;
```

Figura 5.18: Exemplu de tratare a unor excepții

Nu în ultimul rând, în folderul dedicat controllerelor, sunt prezente clasele ce moștenesc interfața de bază „Controller” din pachetul „AspNetCore.Mvc” și care conțin metode pentru fiecare endpoint oferit de serverul platformei UNIVENT.

```
28 | | | | | [HttpGet]  
29 | | | | | 0 references  
30 | | | | | public async Task<IActionResult> GetAllUniversities()  
31 | | | | | {  
32 | | | | |     var query = new GetAllUniversities();  
33 | | | | |     var response = await _mediator.Send(request: query);  
34 | | | | |     var universities = _mapper.Map<List<UniversityResponse>>(source: response);  
35 | | | | |     return Ok(value: universities);  
36 | | | | | }  
37 | | | | |  
38 | | | | | [HttpPost]  
39 | | | | | [Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]  
40 | | | | | [Authorize(Policy = "AdminOnly")]  
41 | | | | | 0 references  
42 | | | | | public async Task<IActionResult> CreateUniversity([FromBody] UniversityCreate univ)  
43 | | | | | {  
44 | | | | |     var command = _mapper.Map<CreateUniversityCommand>(source: univ);  
45 | | | | |     var response = await _mediator.Send(request: command);  
46 | | | | |     var university = _mapper.Map<UniversityResponse>(source: response);  
47 | | | | |     return CreatedAtAction(actionName: nameof(GetUniversityById), routeValues: new { id = response.UniversityID }, value: university);  
48 | | | | | }  
49 | | | | |
```

Figura 5.19: Exemplu de endpoint-uri pentru entitatea Universitate

Ultimul nivel al serverului este Dal, ce poate fi asociat cu stratul de Cadre și Drivere al arhitecturii Clean, având ca rol accesarea informațiilor din baza de date.

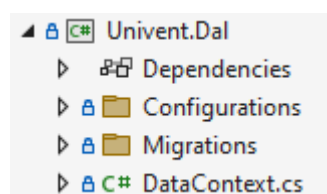


Figura 5.20: Structura nivelului Dal

Acest nivel al serverului definește configurările pentru constrângeri ale bazei de date. Aici este utilizat pachetul „Entity Framework”, ce generează pe baza configurărilor și a modelelor din Domain, clase denumite migrări. Un exemplu sugestiv, este în figura următoare, în care se definesc constrângeri pentru un Eveniment, care conține chei străine pentru tipul de eveniment corespunzător și profilul utilizatorului care a creat acel eveniment.


```

1  using Microsoft.EntityFrameworkCore;
2  using Microsoft.EntityFrameworkCore.Metadata.Builders;
3  using Univent.Domain.Aggregates.EventAggregate;
4
5  namespace Univent.Dal.Configurations
6  {
7      1 reference
8      internal class EventConfig : IEntityTypeConfiguration<Event>
9      {
10         0 references
11         public void Configure(EntityTypeBuilder<Event> builder)
12         {
13             builder.HasOne(navigationExpression: e => e.EventType)
14                 .WithMany(navigationExpression: et => et.Events)
15                 .HasForeignKey(foreignKeyExpression: e => e.EventTypeID);
16
17             builder.HasOne(navigationExpression: e => e.Creator)
18                 .WithMany(navigationExpression: up => up.CreatedEvents)
19                 .HasForeignKey(foreignKeyExpression: e => e.UserProfileID);
20         }
21     }
22 }

```

Figura 5.21: Exemplu de configurare pentru constrângeri asupra unei entități de tip Eveniment

Totodată, pentru generarea migrărilor, este nevoie și de definirea unui context, pentru conectarea la baza de date. Acest context este constituit dintr-o listă cu toate tipurile de entități prezente și configurările ce trebuie aplicate.

```

8  namespace Univent.Dal
9  {
10     90 references
11     public class DataContext : IdentityDbContext
12     {
13         13 references
14         public DbSet<UserProfile> UserProfiles { get; set; }
15         6 references
16         public DbSet<University> Universities { get; set; }
17         6 references
18         public DbSet<Rating> Ratings { get; set; }
19         13 references
20         public DbSet<Event> Events { get; set; }
21         20 references
22         public DbSet<EventParticipant> EventParticipants { get; set; }
23         6 references
24         public DbSet<EventType> EventTypes { get; set; }
25
26         //Constructor, which passes to base class
27         0 references
28         public DataContext(DbContextOptions options) : base(options)
29         {
30         }
31
32         0 references
33         protected override void OnModelCreating(ModelBuilder modelBuilder)
34         {
35             modelBuilder.ApplyConfiguration(configuration: new UserProfileConfig());
36             modelBuilder.ApplyConfiguration(configuration: new RatingConfig());
37             modelBuilder.ApplyConfiguration(configuration: new EventConfig());
38             modelBuilder.ApplyConfiguration(configuration: new EventParticipantConfig());
39
40             modelBuilder.ApplyConfiguration(configuration: new IdentityUserLoginConfig());
41             modelBuilder.ApplyConfiguration(configuration: new IdentityUserRoleConfig());
42             modelBuilder.ApplyConfiguration(configuration: new IdentityUserTokenConfig());
43         }
44     }
45 }

```

Figura 5.22: Contextul bazei de date

5.3 BAZA DE DATE

Pentru implementarea bazei de date, au fost folosite Migrările generate în nivelul Dal al serverului. Acestea au fost generate, cu ajutorul pachetului Entity Framework, utilizând comanda „add-migration” în consola „Package Manager” al framework-ului ASP.NET Core. Migrările conțin două metode principale: Up, utilizată atunci când se dorește aplicarea unei modificări la schema bazei de date, și Down, în caz de revenire la o versiune anterioară sau eliminarea modificărilor aduse schemei bazei de date.

```
53 migrationBuilder.CreateTable(  
54     name: "Universities",  
55     columns: table => new  
56     {  
57         UniversityID = table.Column<Guid>(type: "uniqueidentifier", nullable: false),  
58         Name = table.Column<string>(type: "nvarchar(max)", nullable: false)  
59     },  
60     constraints: table =>  
61     {  
62         table.PrimaryKey(name: "PK_Universities", columns: x => x.UniversityID);  
63     });
```

Figura 5.23: Exemplu de cod pentru creare a unei tabele, în migrare

Tot cu ajutorul migrărilor, au fost definite două roluri prestabilite: admin și user. De asemenea, în migrări a fost introdus manual și un cont implicit, ce are rolul de admin. Întrucât la inițierea unei baze de date, sunt executate toate migrările în ordine, acest lucru va determina adăugarea forțată a contului de administrator, în mod automat. Pe de altă parte, rolul de utilizator este atribuit automat la crearea unui cont, în metoda de Handler al endpoint-ului de Register.

După definirea migrărilor, acestea sunt executate cu comanda „update-database”, care va verifica toate migrările disponibile și le va executa pe cele care nu au fost executate până la momentul respectiv.

Utilizând pachetul Entity Framework și funcționalitatea de migrări pentru generarea codului SQL corespunzător, abordarea utilizată în crearea bazei de date este „code-first”. Această strategie este ideală pentru întreținerea ușoară a sistemului și actualizarea sa, în cazul schimbării cerințelor, întrucât dezvoltatorii pot crea migrări noi și pot actualiza structura bazei de date printr-o simplă comandă.

Astfel, în figura de mai jos, este o schemă de ansamblu a tuturor tabelor din baza de date, împreună cu câmpurile necesare și constrângerile de chei primare și străine. Tabelele utilizate de baza de date a platformei UNIVENT sunt: „UserProfiles” (pentru informațiile utilizatorilor), „Universities”, „Events”, „EventTypes” (pentru tipurile de evenimente acceptate), „EventParticipants” (pentru combinațiile de evenimente și utilizatori, ce definesc participanții), „Ratings” (pentru toate recenziile utilizatorilor) și încă trei tabele generate în mod automat de pachetul Identity, utilizat în partea de server. Aceste tabele sunt: „Users” (pentru conturile utilizatorilor), „Roles” și „UserRoles” (pentru combinațiile de utilizatori și rolul fiecăruia).

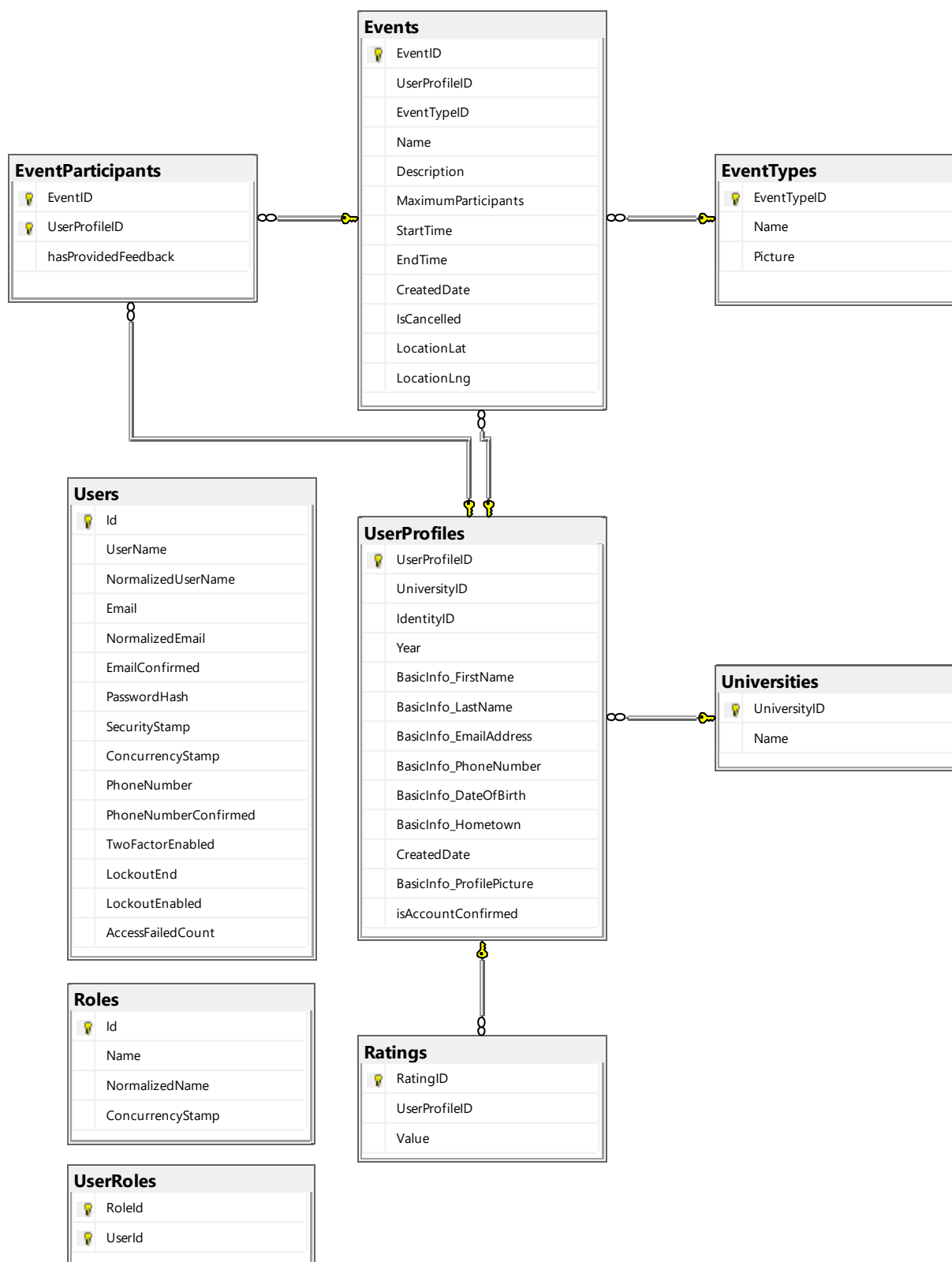


Figura 5.24: Diagrama bazei de date

Pentru a reține într-un mod securizat conturile utilizatorilor de pe platformă, pachetul Identity aplică în mod automat tuturor parolelor, un algoritm de hash criptografic puternic. Acest algoritm folosit implicit este bcrypt, ce transformă parolele în hash-uri, adică șiruri de

caractere, ce par a fi aleatorii la prima vedere. Aceste șiruri de caractere nu sunt stocate într-o formă ușoară de citit, oferind astfel un nivel suplimentar de securitate, însă nu pot fi inversate pentru a recupera parolele originale. Bcrypt este un algoritm rezistent la atacurile de tip bruteforce, fiind dificil pentru atacatori să inverseze sau să ghicească parolele originale. Pentru a crea un hash, algoritmul colectează parola utilizatorului atunci când acesta își creează contul sau își schimbă parola. Acest hash este stocat în baza de date, iar dacă utilizatorul își schimbă parola, hash-ul nou este comparat cu cel stocat în baza de date. Dacă cele două hash-uri coincid, autentificarea este reușită, altfel este respinsă [21].

Modelul de ștergere al datelor, pentru tabelele ce sunt legate prin constrângeri de chei străine, este cel implicit, în cascadă, cu excepția tabelii „EventParticipants”, ce conține o cheie primară compusă din două coloane. Astfel, ștergerea datelor din această tabelă este implementată manual, în metodele de tip Handler, care pot declanșa acest scenariu. Totodată, tabelele generate de pachetul Identity conțin câmpuri pentru identificatori unici din alte tabele, însă nu au fost implementate constrângeri pentru chei străine. Din acest motiv, tabela UserProfiles conține un câmp denumit „IdentityID” și cazurile de adăugare, editare și ștergere au fost implementate manual.

5.4 CLIENTUL WEB

În dezvoltarea clientului Front-End, a fost utilizată biblioteca React, împreună cu pachete adiționale. Lista pachetelor adiționale, împreună cu versiunile acestora, se află în fișierul „package.json”. De asemenea, pachetele în sine, sunt instalate local, în dosarul „node_modules”. Astfel, în figura de mai jos, este ilustrată structura folderelor și fișierelor ce constituie proiectul React.

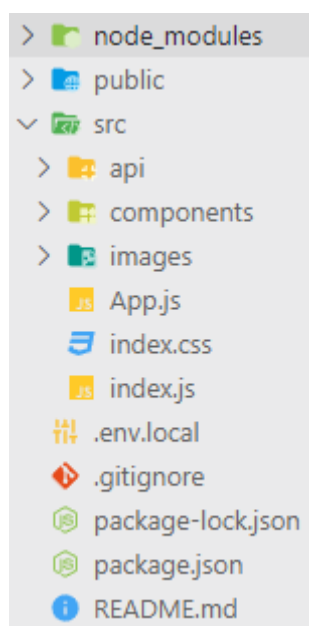


Figura 5.25: Structura principală a clientului web

În dosarul „components” se regăsesc componentele generice utilizate și paginile aplicației UNIVENT. În fiecare dosar de componentă sau pagini, sunt prezente fișiere JavaScript ce definesc comportamentul sistemului și opțional fișiere de tip CSS pentru stilizarea aplicației.

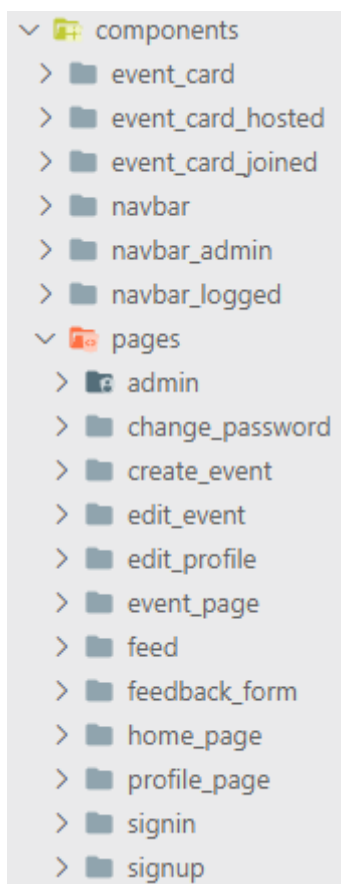


Figura 5.26: Dosarul „components” al proiectului React

Pentru implementarea componentelor din paginile clientului, a fost folosit pachetul „Material-UI”, librărie ce a fost dezvoltată de Facebook și care oferă o gamă largă de șabloane pentru componente generice. Acestea au fost pe urmă personalizate, în funcție de cerințele platformei UNIVENT. Câteva exemple de componente „Material-UI” populare, sunt: „Grid”, „Button”, „Dialog”, „FormControl”, „Avatar”, „Rating”, „Typography”, sau „Paper”. În plus, „Material-UI” pune la dispoziție și un set de iconițe (numit „Material Icons”), convertite în format SVG și pregătite a fi utilizate direct, după importare [22].

De asemenea, pentru validarea datelor introduse de utilizator, s-a folosit pachetul „yup”. Pentru a îmbunătăți platforma, datele sunt validate și în partea de client, înainte de a fi trimise către server. Acest pachet permite atât utilizarea validatoarelor prestabilite, cât și introducerea de reguli de validare personalizate.

Pentru a oferi o securitate îmbunătățită, a fost utilizat un pachet adițional, numit „jwt-decoder”. Acesta are rolul de a decodifica cheia unică primită din partea serverului, pentru a extrage informații esențiale despre utilizatorul autentificat, cum ar fi identificatorul unic al contului, identificatorul unic al profilului de utilizator, sau adresa sa de e-mail.

Platforma UNIVENT pune la dispoziție și o hartă interactivă, pentru a marca și vizualiza ușor locația evenimentelor. Acest lucru a fost posibil, prin utilizarea serviciului API de la Google, numit „Maps API”, disponibil pe site-ul „Google Cloud Platform”. Prin crearea unui cont pe platforma Google, a fost selectat serviciul Google Maps, iar ca rezultat, a fost generată o cheie unică pentru utilizarea acestui API, în mod securizat. Cheia corespunzătoare se află în fișierul local „.env.local”, întrucât rolul acestui fișier este de a stoca date cu caracter confidențial. Alte servicii Google, utilizate în corelație cu Maps, sunt „Places API” (pentru autocompletarea locațiilor, în bara de căutare) și „Geocoding API”/„Geolocation API” (pentru extragerea coordonatelor).

Nu în ultimul rând, comunicarea dintre client și server a fost facilitată de pachetul „Axios”. Cu ajutorul acestei biblioteci JavaScript, au fost configurate toate tipurile de cereri HTTP, cu rutele corespunzătoare. Aceste rute trebuiesc precizate, conform modului în care au fost definite pe partea de server. Pe de altă parte, această bibliotecă este folosită și pentru a interpreta rezultatele primite din partea serverului. Răspunsul este reținut implicit într-o variabilă de tip „response”, ce conține trei componente principale: „data”, „status” și „error”. Datele returnate de server sunt în format JSON, iar „Axios” le încapsulează într-un obiect JavaScript, numit „response”. În acest fel, prin „response.data” se pot accesa datele returnate, prin „response.status” se poate verifica codul returnat, iar în caz de eroare, „response.error” oferă detalii suplimentare.

6. CONFIGURARE ȘI UTILIZARE

6.1 INSTALARE ȘI CONFIGURARE

Pentru rularea aplicației UNIVENT, este necesară instalarea framework-urilor și bibliotecilor pentru cele trei componente principale: server, baza de date și clientul web.

În primul rând, pentru server, este necesară instalarea kit-ului de dezvoltare software .NET SDK. Versiunea x64, poate fi descărcată în mod gratuit, accesând pagina oficială <https://dotnet.microsoft.com/en-us/download>. După instalarea cu succes a kit-ului .NET, se introduce instrucțiunea „*dotnet tool install --global dotnet-ef*” în linia de comandă, pentru a instala pachetul Entity Framework (mai multe detalii pot fi găsite la adresa <https://learn.microsoft.com/en-us/ef/core/get-started/overview/install>).

Următorul pas îl reprezintă instalarea aplicației Microsoft SQL Server, pentru crearea bazei de date. Aceasta se poate descărca și instala de pe pagina <https://www.microsoft.com/en-us/sql-server/sql-server-downloads>. Odată instalate Entity Framework și SQL Server, trebuie apelată comanda „*dotnet build*” pentru a crea un build, iar apoi comanda „*dotnet ef database update*” pentru a executa toate migrările în ordine, ce vor crea tabelele și câmpurile bazei de date, împreună cu constrângerile necesare. În final, pentru rularea serverului, se apelează linia „*dotnet run*”. De menționat este faptul că prin rularea acestei serii de comenzi, toate pachetele necesare vor fi instalate automat.

Pentru partea de client, este necesară instalarea mediului de execuție „Node.js”, ce poate fi descărcat la adresa <https://nodejs.org/en/download>. Acesta include manager-ul de pachete „npm”, utilizat pentru a instala pachetele necesare proiectului React. Acest lucru poate fi realizat prin execuția instrucțiunii „*npm install*” în linia de comandă, la nivelul dosarului „web-client”. După ce au fost instalate aceste module, rularea clientului se poate realiza prin linia „*npm start*”.

Dacă se dorește evitarea introducerii comenzilor în terminal, ca alternativă se pot utiliza gratuit mai multe IDE-uri, special optimizate pentru dezvoltarea în ASP.NET Core și JavaScript. Ca exemplu, ar fi suita Visual Studio: Microsoft Visual Studio 2022 Community Edition, pentru server (oferă atât un set preconfigurat de instrumente, pentru ASP.NET Core, ce instalează automat SDK-ul, cât și un manager de pachete, denumit „NuGet”) și Microsoft Visual Studio Code, pentru clientul web.

6.2 MANUAL DE UTILIZARE

Aplicația UNIVENT poate fi folosită de către două tipuri de utilizatori: studenți înrolați la universitățile partenere și administrator de sistem. Platforma pune la dispoziție o serie de funcționalități diferite pentru aceste roluri, singura parte comună fiind autentificarea în cont. După rularea aplicației, utilizatorul este întâmpinat, cu pagina principală, pe nume „Landing Page”.

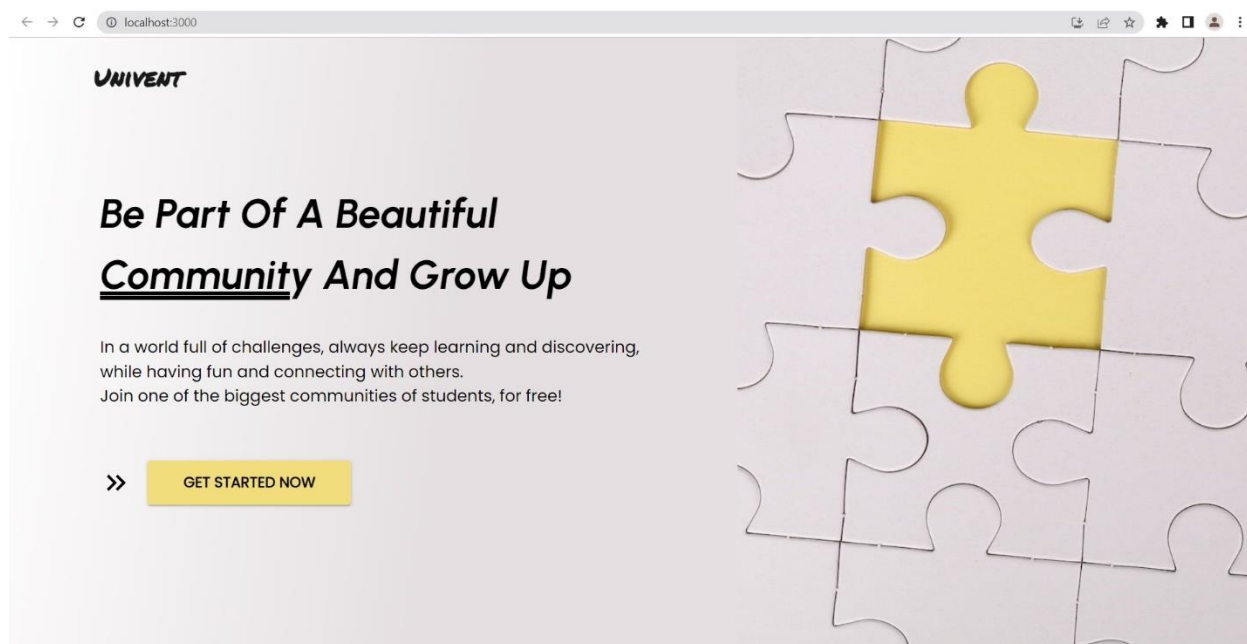


Figura 6.1: Pagina principală „Landing Page”

La apăsarea pe butonul „GET STARTED NOW”, utilizatorul va fi redirecționat pe pagina de autentificare în cont.

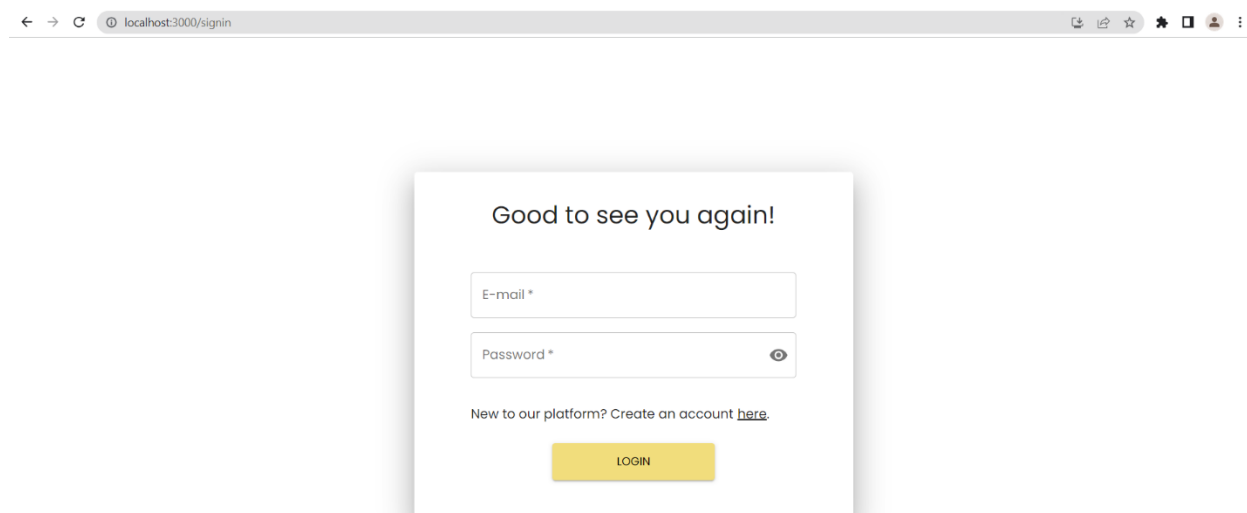


Figura 6.2: Pagina de autentificare în cont

Pe această pagină, utilizatorul poate introduce adresa de e-mail și parola, pentru a intra în cont. De asemenea, dacă studentul încă nu are cont pe platformă, acesta poate apăsa pe link-ul subliniat pentru a fi redirecționat pe pagina de înregistrare.

Sign Up

1 2 3

First Name * Catalin

Last Name * Secosan

University * Universitatea Politehnica Timi...

Year * IV

Phone number * 0773501925

Date of birth * 17-03-2000

Hometown * Timisoara

Already have an account? [Sign In](#)

NEXT STEP

Figura 6.3: Pagina de înregistrare – pasul 1

Sign Up

1 2 3

First Name * Catalin33

Last Name * Secosan

Must be only letters

University * Universitatea Politehnica Timi...

Year * IV

Phone number * 07735

Invalid phone number format

Date of birth * 17-03-1960

Age should be between 18 and 26

Hometown * Timisoara

Already have an account? [Sign In](#)

NEXT STEP

Figura 6.4: Exemplu de validare a datelor pe pagina de înregistrare

Sign Up

1 2 3

First Name *
Catalin

Last Name *
|

University *
Universitatea Politehnica Timișoara

Please fill out this field.

Figura 6.5: Exemplu de date obligatorii de completat

După completarea informațiilor la primul pas, utilizatorul este redirecționat către pasul al doilea.

← → ↻ localhost:3000/register

Sign Up

✓ 2 3

E-mail *
catalin_secosan@email.com

Password *
.....

Confirm password *

Password must have at least:

- 8 characters
- one lowercase character
- one uppercase character
- one digit
- one special character !?~

BACK NEXT STEP

Figura 6.6: Pagina de înregistrare – pasul 2

Primii doi pași sunt obligatorii de completat, pe când ultimul vizează adăugarea pozei de profil, pas ce este opțional. De asemenea, utilizatorul are opțiunea de a se întoarce la unul din pașii precedenți, dacă are nevoie.

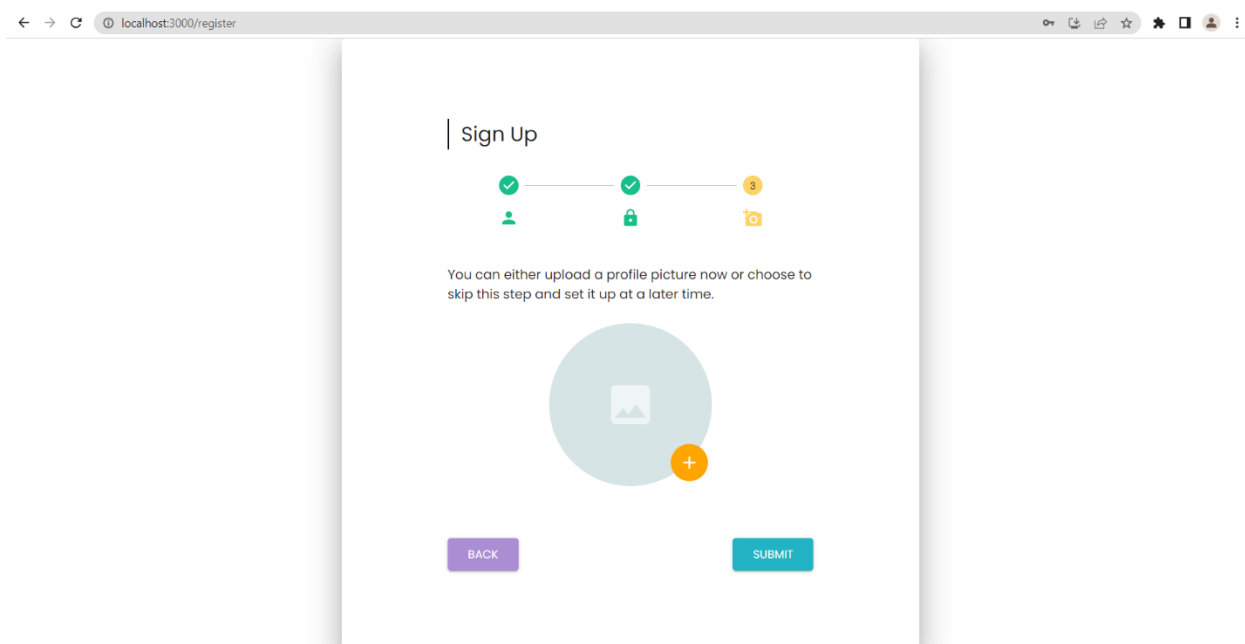


Figura 6.7: Pagina de înregistrare – pasul 3

După confirmarea informațiilor, acestea vor fi trimise la server, creându-se un cont nou pt acest utilizator (sistemul îi va atribui acestuia în mod automat rolul de utilizator simplu/student). Cu toate acestea, administratorul de sistem va revizui informațiile utilizatorului și va confirma contul într-un interval de maximum 24 de ore. Astfel, utilizatorul nou va putea accesa contul după ce administratorul îl va aproba pe platformă.

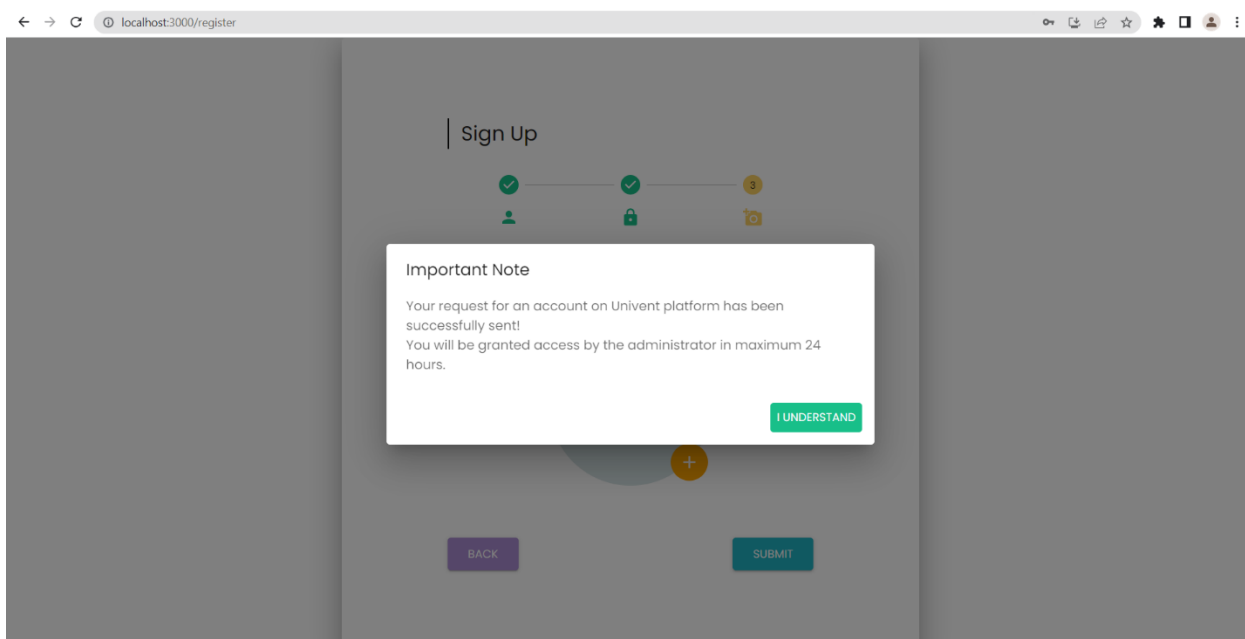
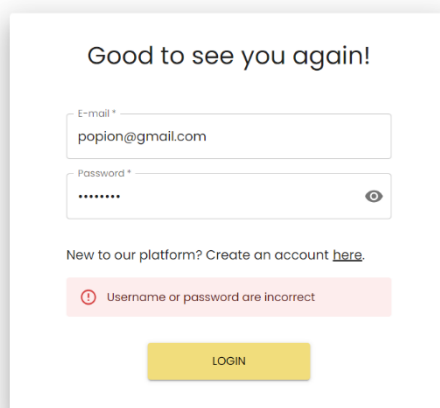


Figura 6.8: Notificare legată de aprobarea pe platformă



Good to see you again!

E-mail *
popion@gmail.com

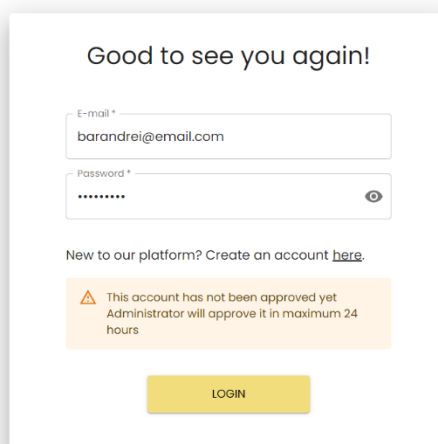
Password *

New to our platform? Create an account [here](#).

❗ Username or password are incorrect

LOGIN

Figura 6.9: Exemplu de eroare pentru credențiale incorecte



Good to see you again!

E-mail *
barandrei@email.com

Password *

New to our platform? Create an account [here](#).

⚠ This account has not been approved yet
Administrator will approve it in maximum 24
hours

LOGIN

Figura 6.10: Studentul încă nu a fost confirmat pe platformă

În cazul în care utilizatorul a reușit să se autentifice în cont și a fost autorizat de către sistem, utilizatorul va fi redirecționat pe pagina principală aferentă rolului curent. În subcapitolele ce urmează, vor fi detaliate funcționalitățile disponibile fiecărui tip de utilizator.

6.2.1 FUNCȚIONALITĂȚI STUDENT

După autentificarea în cont și autorizarea din partea sistemului, studentul va fi redirecționat pe pagina de utilizator simplu. Această pagină poartă denumirea de Feed, unde vor fi prezentate evenimentele disponibile. Studentul are opțiunea de a filtra evenimentele după tip, de a căuta evenimente după nume și de a accesa detaliile unui eveniment, apăsând pe butonul „CHECK MORE DETAILS”, al evenimentului în cauză.

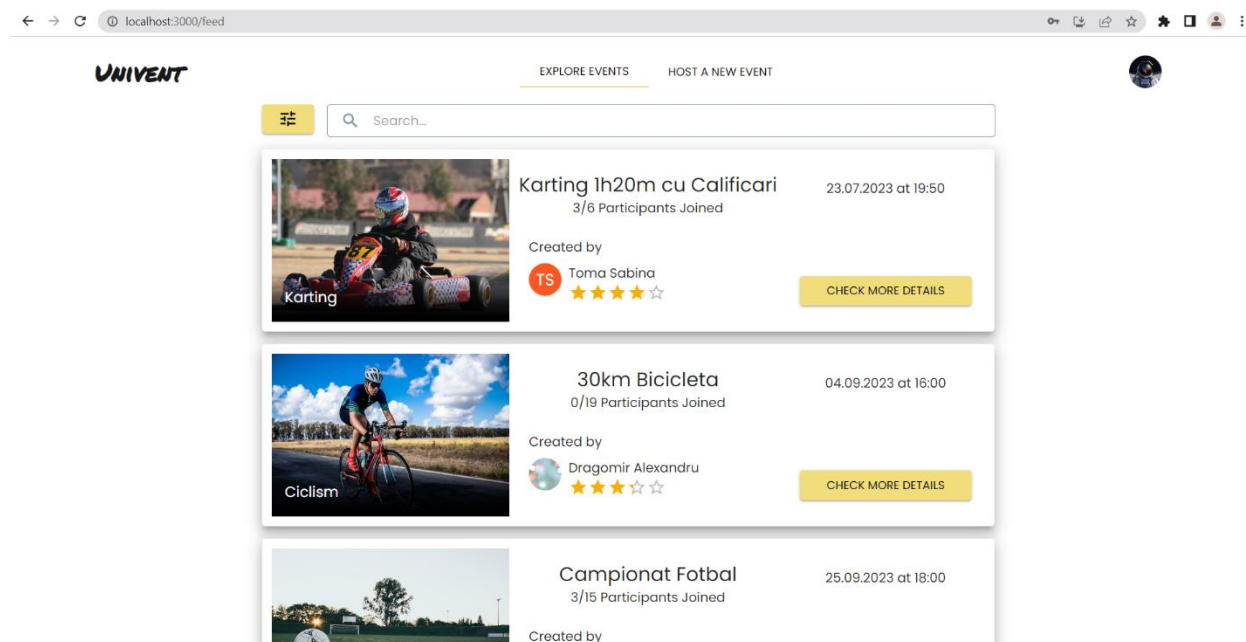


Figura 6.11: Pagina „Feed”

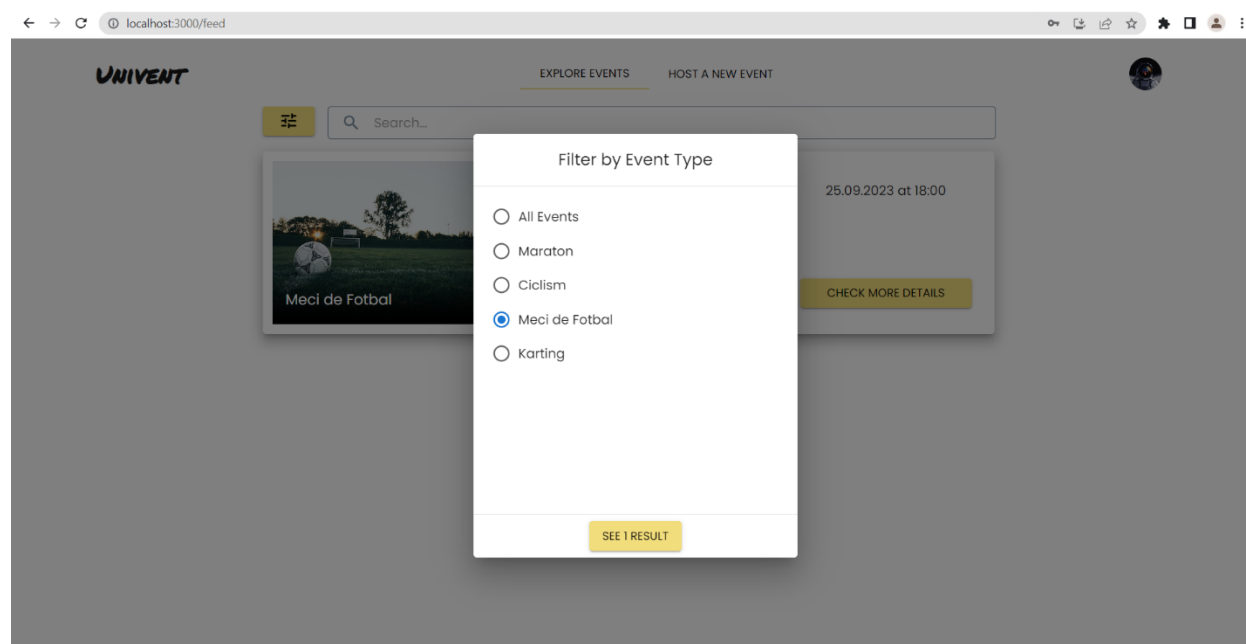


Figura 6.12: Opțiunea de filtrare evenimente

În cazul în care nu sunt evenimente disponibile pentru tipul de evenimente ales, butonul „SEE ... RESULTS” va fi înlocuit de textul „No events available at the moment”.

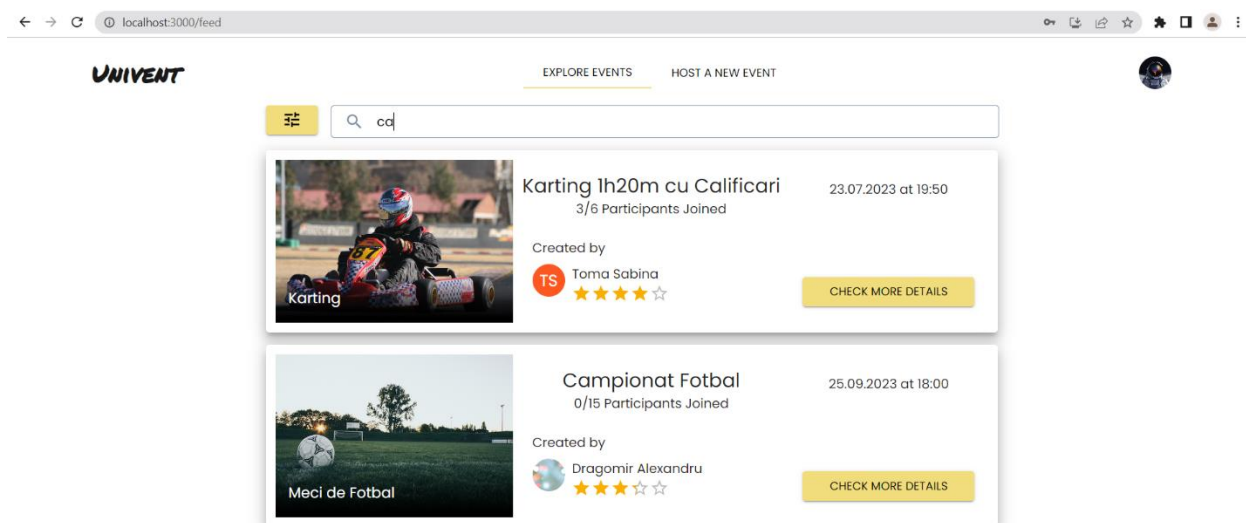


Figura 6.13: Opțiunea de căutare după numele evenimentelor

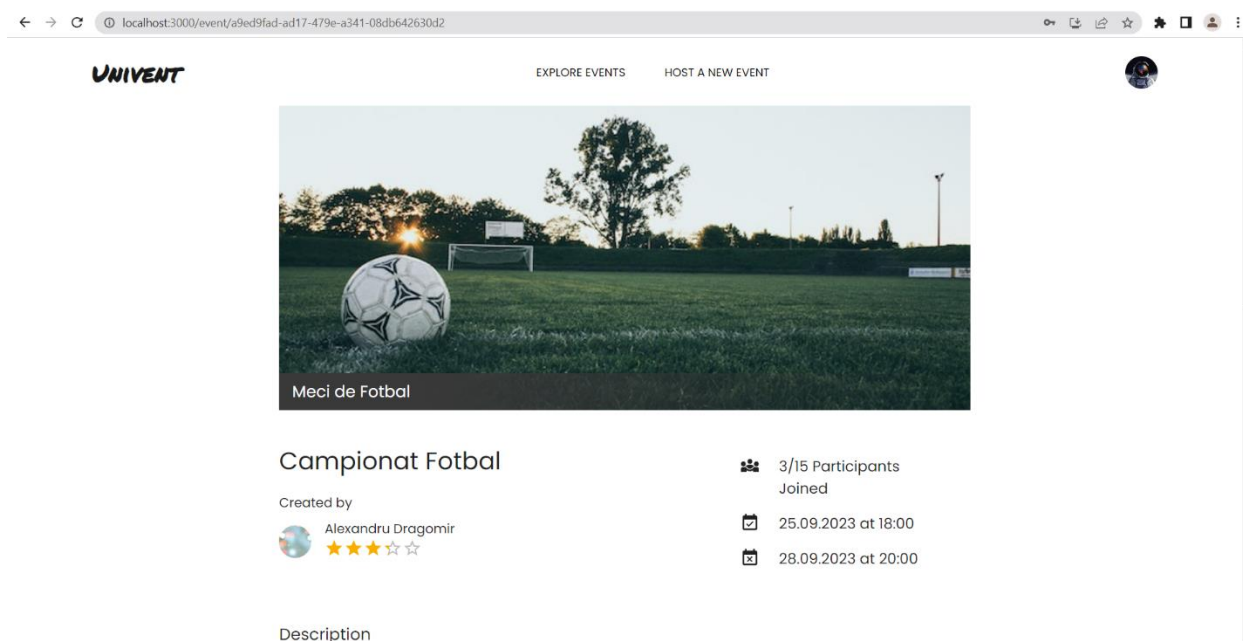


Figura 6.14: Pagina cu detaliile unui eveniment

În figura de mai sus, este un exemplu de o pagină a unui eveniment existent. Aici, studentul poate verifica detalii ale evenimentului și se poate înscrie, dacă încă nu a făcut-o și dacă numărul maxim de participanți nu a fost atins încă.

UNIVENT

EXPLORE EVENTS HOST A NEW EVENT



Meci de Fotbal

Campionat Fotbal

Created by

Alexandru Dragomir

★★★★☆

3/15 Participants
Joined

25.09.2023 at 18:00

28.09.2023 at 20:00

Description

Odata cu inceperea facultatii, vom tine campionat de fotbal zilnic. Meciurile se vor desfasura in perioada 25-28 Septembrie, intre orele 18-20.

Location on the map

CHECK ENROLLED PARTICIPANTS

Figura 6.15: Pagina cu detaliile unui eveniment – Continuare

EXPLORE EVENTS HOST A NEW EVENT

RECENTER

List of Enrolled Participants

Georgescu Ana

★★★★☆

Matei Liliana

★★★★☆

Pop Ion

★★★★☆

CHECK ENROLLED PARTICIPANTS

Figura 6.16: Lista de participanți, pentru un eveniment

În bara de navigare, poziționată în partea de sus a ecranului, se află două opțiuni. Selectarea primei opțiuni, „EXPLORE EVENTS”, va redirecționa utilizatorul pe pagina de Feed, iar a doua opțiune, numită „HOST A NEW EVENT” va da studentului posibilitatea de a crea un eveniment nou.

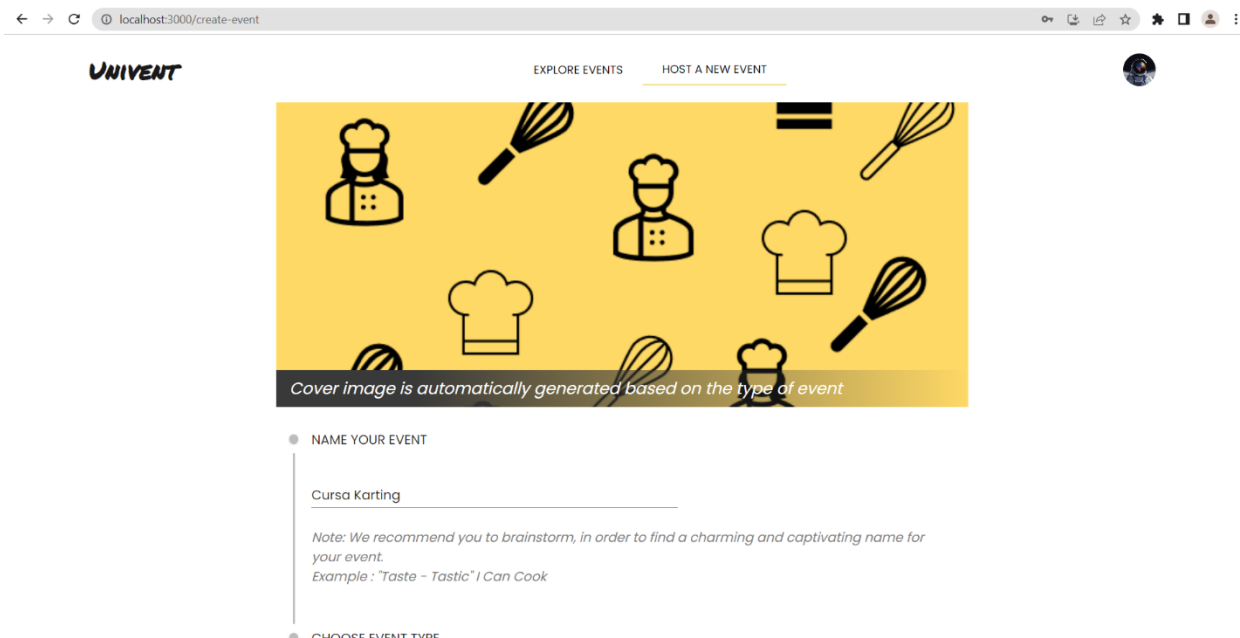


Figura 6.17: Crearea unui eveniment nou

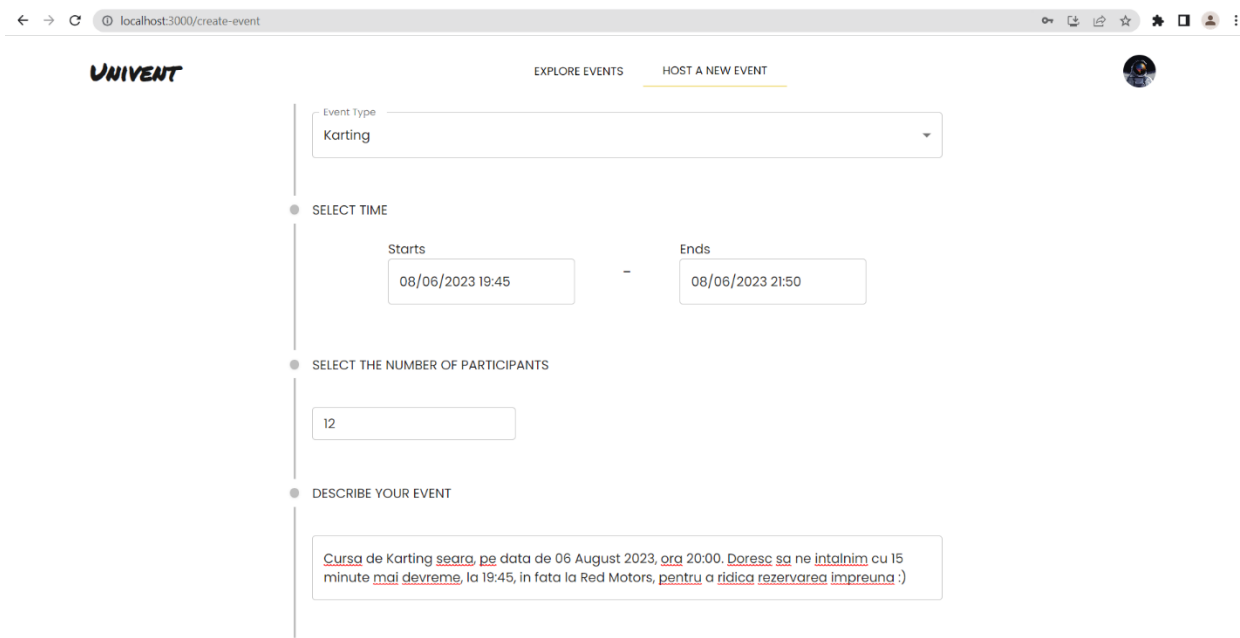


Figura 6.18: Crearea unui eveniment nou – continuare

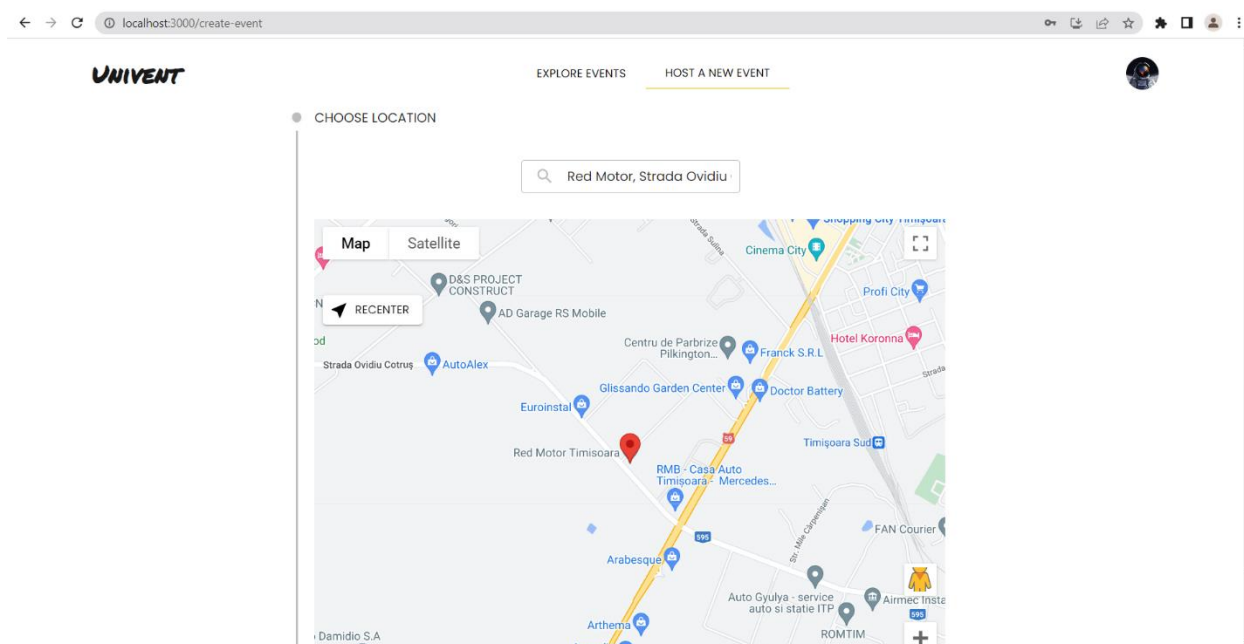


Figura 6.19: Crearea unui eveniment nou – final

Tot în bara de navigare, utilizatorul poate accesa un meniu cu opțiuni adiționale, apăsând pe avatarul din colțul dreapta sus. Această acțiune va deschide un meniu format din două setări: „Profile”, pentru a accesa profilul personal și „Logout” pentru a deconecta utilizatorul de pe platformă.

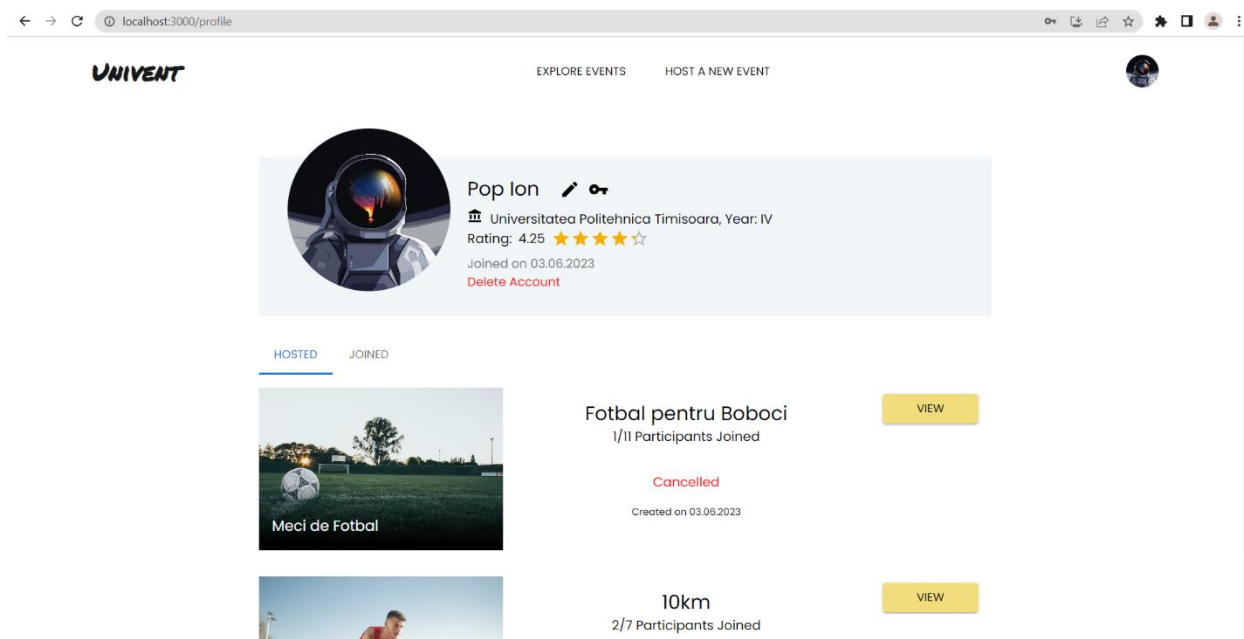


Figura 6.20: Pagina de profil a utilizatorului

Pagina de profil oferă opțiuni de editare a informațiilor personale, schimbare de parolă, ștergerea permanentă a contului și câte un istoric pentru evenimentele create și participările la evenimente create de alți studenți.

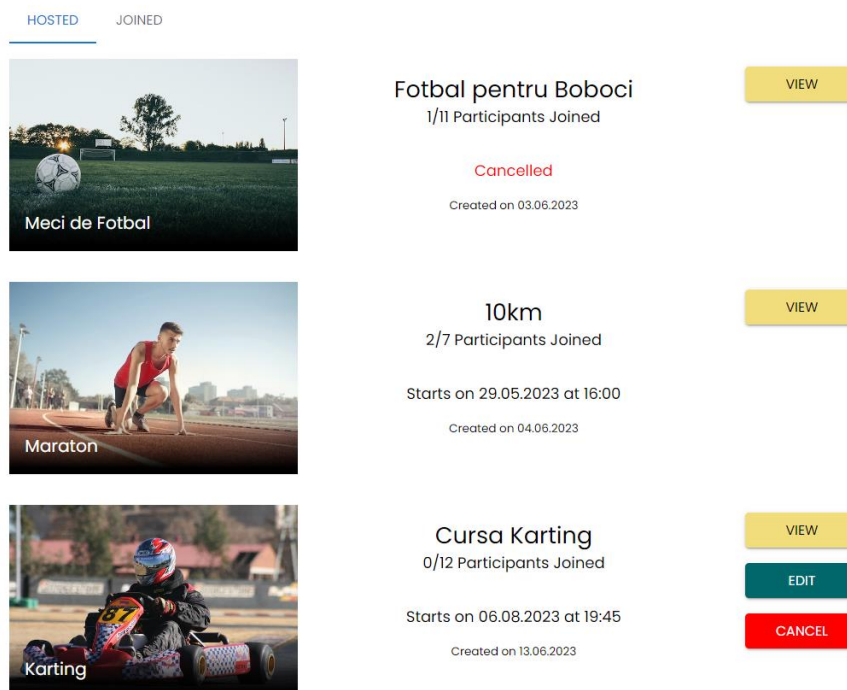


Figura 6.21: Istoric al evenimentelor create

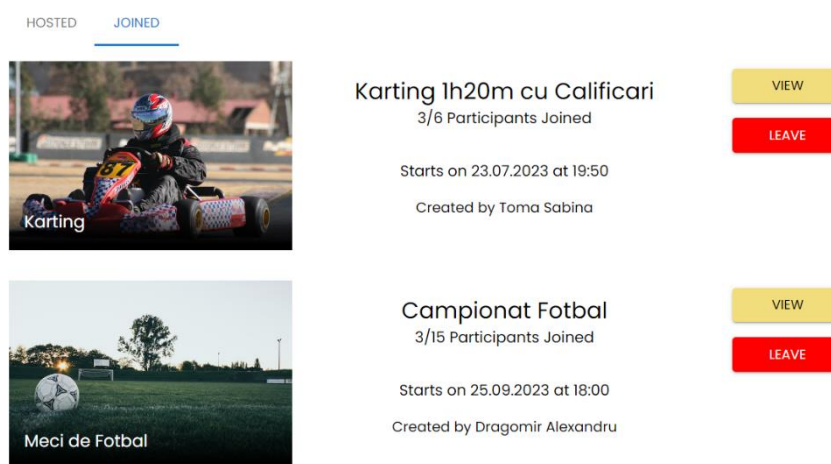


Figura 6.22: Istoric al participărilor la alte evenimente

În cazul unui eveniment creat, studentul are opțiunea de a edita informațiile acestuia sau de a-l anula definitiv, dacă evenimentul încă nu a avut loc. În cazul editării unui eveniment, utilizatorul apasă pe butonul „EDIT” și va fi redirecționat pe o pagină similară cu cea a creării unui eveniment nou. De această dată însă, informațiile sunt implicit completate. Pe de altă parte, pentru a anula un eveniment, utilizatorul apasă pe butonul „CANCEL” și va apărea un mesaj ce necesită confirmarea pentru a finaliza anularea.

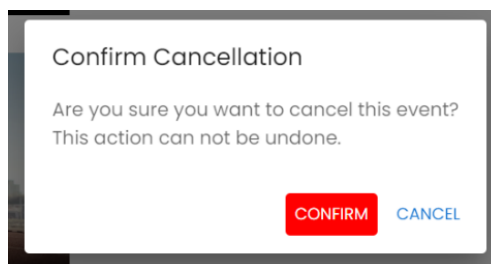


Figura 6.23: Fereastra de confirmare pentru a anula un eveniment

În cazul istoricului participărilor la alte evenimente, utilizatorul are opțiunea de a renunța la participarea unui eveniment care nu a avut loc, încă. În mod similar cu anularea unui eveniment, această acțiune necesită confirmarea studentului. În schimb, dacă un student a părăsit un eveniment, acesta se poate înrola din nou, dacă evenimentul mai este disponibil și limita de participanți nu a fost încă atinsă.

Pentru a edita informațiile personale, utilizatorul poate apăsa pe iconița , din dreptul numelui de utilizator. În schimb, pentru schimbarea parolei, studentul poate apăsa pe cea de-a doua iconiță .

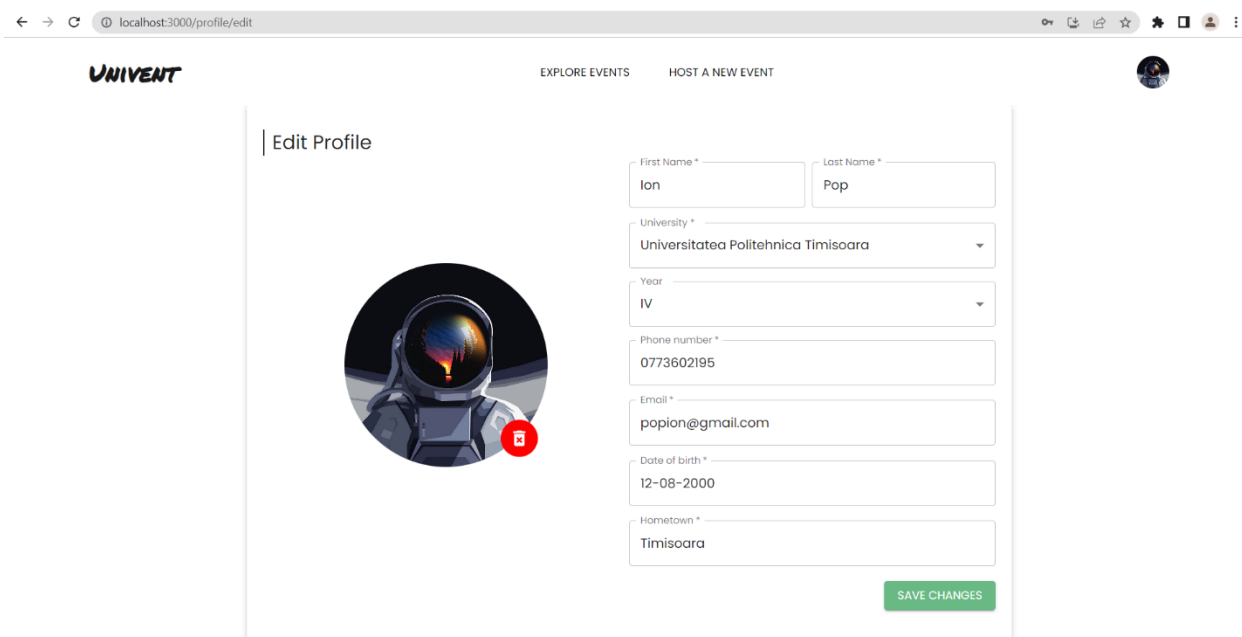


Figura 6.24: Pagina de editare a informațiilor profilului

Cu ajutorul paginii de editare a informațiilor personale, utilizatorul poate să editeze orice informație, dar și să adauge sau să șteargă poza de profil.

← → localhost:3000/profile/change-password

UNIVENT EXPLORE EVENTS HOST A NEW EVENT

Change password

Old Password *

Password *

Confirm password *

Password must have at least:

- 8 characters
- one lowercase character
- one uppercase character
- one digit
- one special character .!?-

SAVE CHANGES

Figura 6.25: Pagina de schimbare a parolei

Pentru acțiunea de schimbare a parolei, platforma UNIVENT impune introducerea parolei vechi, din motive de securitate. După introducerea parolei vechi și a parolei noi, utilizatorul va fi întâmpinat de o fereastră, pentru a confirma cererea de schimbare a parolei. Dacă parola veche este corectă, se va actualiza parola nouă și utilizatorul va fi deconectat din cont, în mod automat. Pentru a continua navigarea pe site, studentul va trebui să se autentifice în cont din nou, cu parola nouă. Pe de altă parte, dacă parola veche este incorectă, va fi afișat un mesaj de eroare.

Change password

Old Password *

Password *

Confirm password *

Password must have at least:

- 8 characters
- one lowercase character
- one uppercase character
- one digit
- one special character .!?-

❗ The old password you provided is not correct

SAVE CHANGES

Figura 6.26: Mesajul de eroare afișat, dacă parola veche este incorectă

Dacă se dorește ștergerea contului, utilizatorul poate apăsa pe opțiunea „DELETE ACCOUNT”, cu text roșu. Această acțiune va necesita o confirmare din partea utilizatorului, în mod similar cu opțiunile de anulare sau părăsire a unui eveniment, prezentate mai sus.

Nu în ultimul rând, din istoricul de participări la evenimente, se pot trimite recenzii altor utilizatori. Pentru evenimentele care au avut loc cu succes, fără a fi anulate, un participant poate deschide un formular de exprimare a părerii, prin acordarea de calificative de la 0 la 5, cu increment de 0,5. Acest formular este opțional și poate fi deschis de câte ori se dorește, dar poate fi completat o singură dată. După trimiterea notelor, utilizatorul nu mai poate accesa formularul pentru acel eveniment. Utilizatorul poate accesa formularul de feedback, prin apăsarea opțiunii „Enjoyed this event? Rate other participants here!” de culoare portocalie.

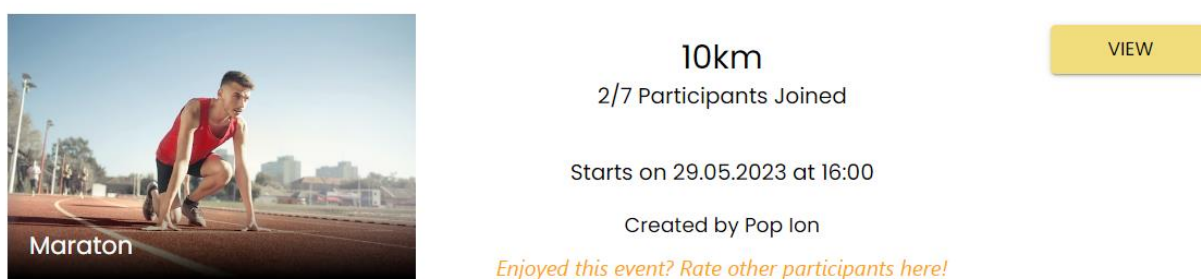


Figura 6.27: Accesarea formularului de feedback

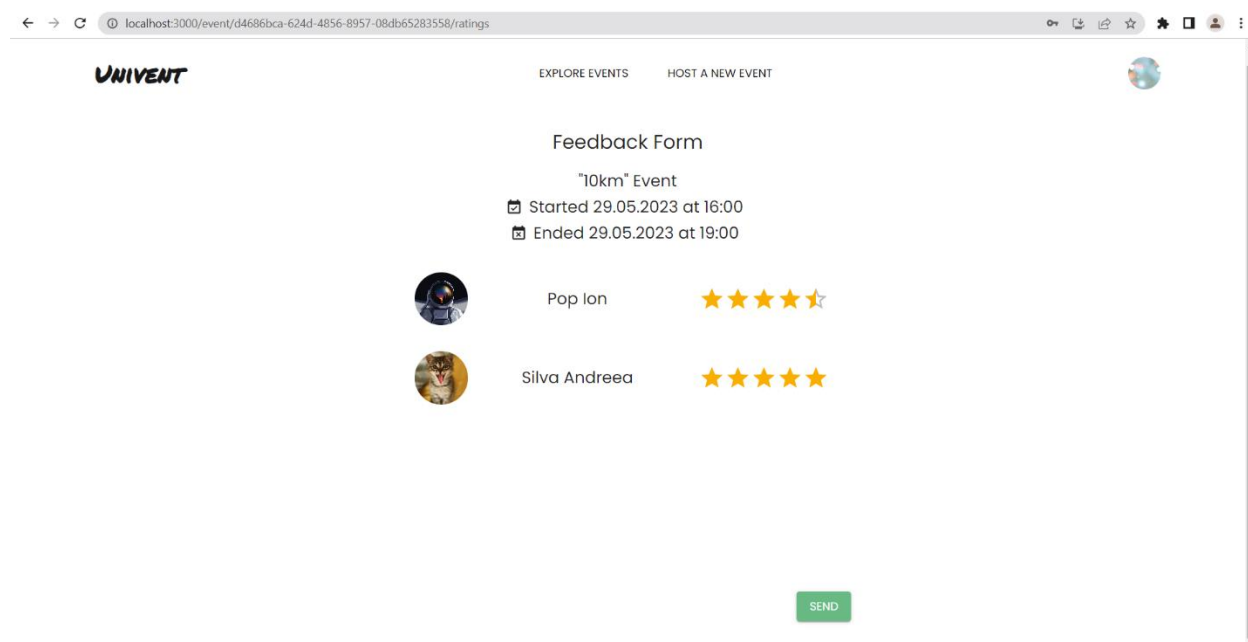


Figura 6.28: Exemplu de completare a formularului de feedback

Pentru completarea acestui formular, utilizatorul trebuie să confirme datele introduse.

6.2.2 FUNCȚIONALITĂȚI ADMINISTRATOR

După autentificarea în cont și autorizarea din partea sistemului, administratorul de sistem va fi redirecționat pe pagina corespunzătoare. Această pagină poartă denumirea de Admin Dashboard. Prin intermediul acestei pagini, administratorul va avea acces la un set de opțiuni exclusive pentru gestionarea platformei. Setările permise sunt împărțite în trei categorii principale: „Universities”, „Event Types” și „Pending Users”. Prima opțiune oferă un meniu pentru vizualizarea universităților existente, adăugarea de universități noi și editarea sau ștergerea celor existente. În mod similar, meniul „Event Types” oferă aceleași categorii de setări, pentru tipurile de evenimente ale platformei. Ultima opțiune, „Pending Users”, îi permite administratorului să revizuiască informațiile conturilor noi de studenți, pentru a aproba acești utilizatori pe platformă.

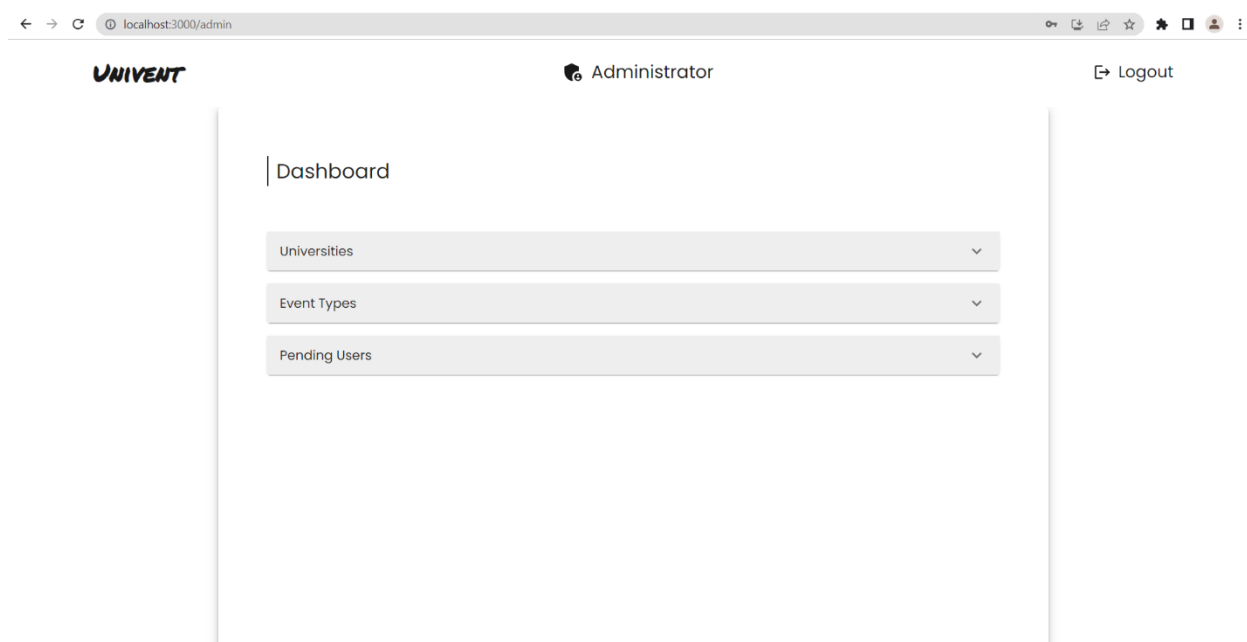


Figura 6.29: Pagina principală a administratorului

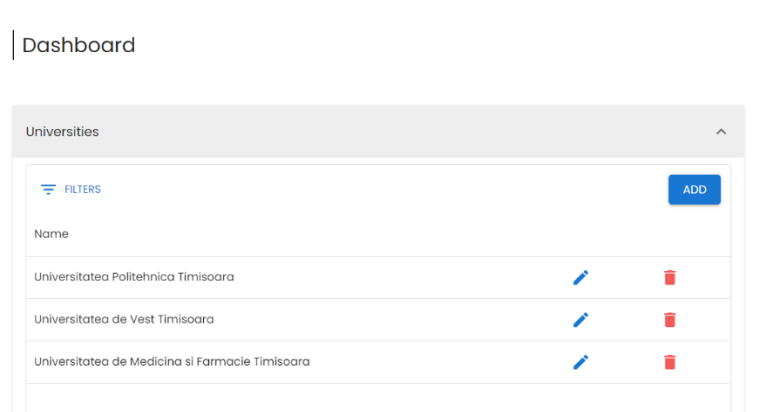


Figura 6.30: Opțiunile disponibile pentru Universități

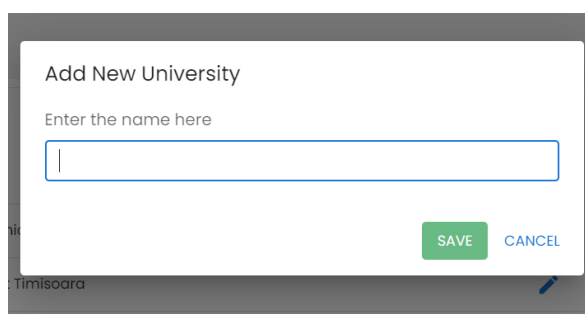


Figura 6.31: Adăugarea unei universități noi, accesibilă prin apăsarea butonului „ADD”

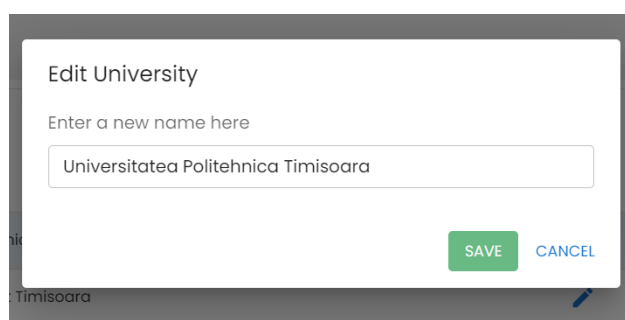


Figura 6.32: Editarea unei universități noi, accesibilă prin apăsarea iconiței 

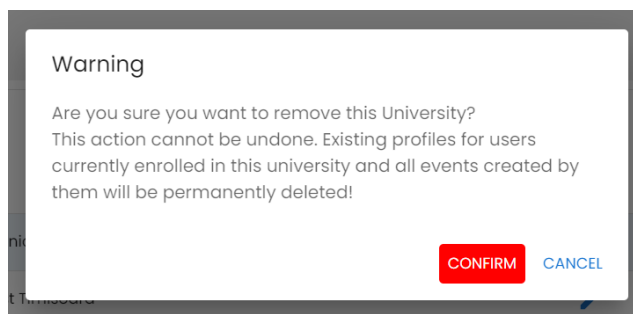


Figura 6.33: Confirmarea ștergerii unei universități

Acțiunile pentru gestionarea tipurilor de evenimente sunt similare cu cele ale universităților, cu diferența că un tip de eveniment necesită un câmp adițional pentru numele pozei unui tip de eveniment. Pentru adăugarea unui tip de eveniment nou, administratorul va trebui să adauge mai întâi poza tipului de eveniment în proiectul React. Directorul destinație pentru aceste imagini poartă numele de „images” și se găsește în folderul „web-client”, iar apoi în dosarul „src”. În stadiul actual al aplicației, acest lucru este necesar pentru a oferi cea mai bună experiență studenților, prin utilizarea de imagini sugestive a tipurilor de evenimente acceptate pe platformă.

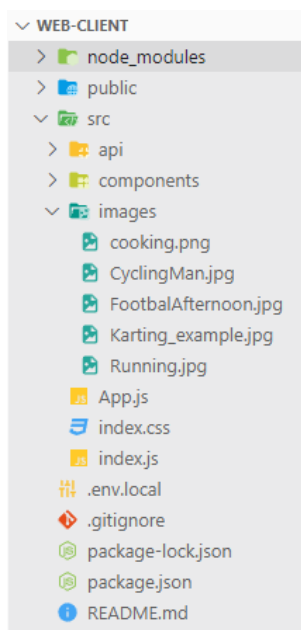
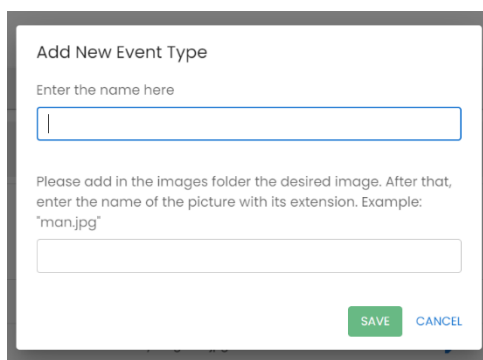


Figura 6.34: Directorul de imagini pentru tipurile de evenimente



Add New Event Type

Enter the name here

Please add in the images folder the desired image. After that, enter the name of the picture with its extension. Example: "man.jpg"

SAVE CANCEL

Figura 6.35: Directorul de imagini pentru tipurile de evenimente

În cele din urmă, cea de-a treia opțiune disponibilă administratorului de sistem este revizuirea informațiilor conturilor noi și aprobarea acestor studenți. După verificarea datelor, administratorul poate aproba un cont nou prin apăsarea iconiței verzi și confirmarea pe fereastra finală. Mai multe detalii se regăsesc în figurile următoare.

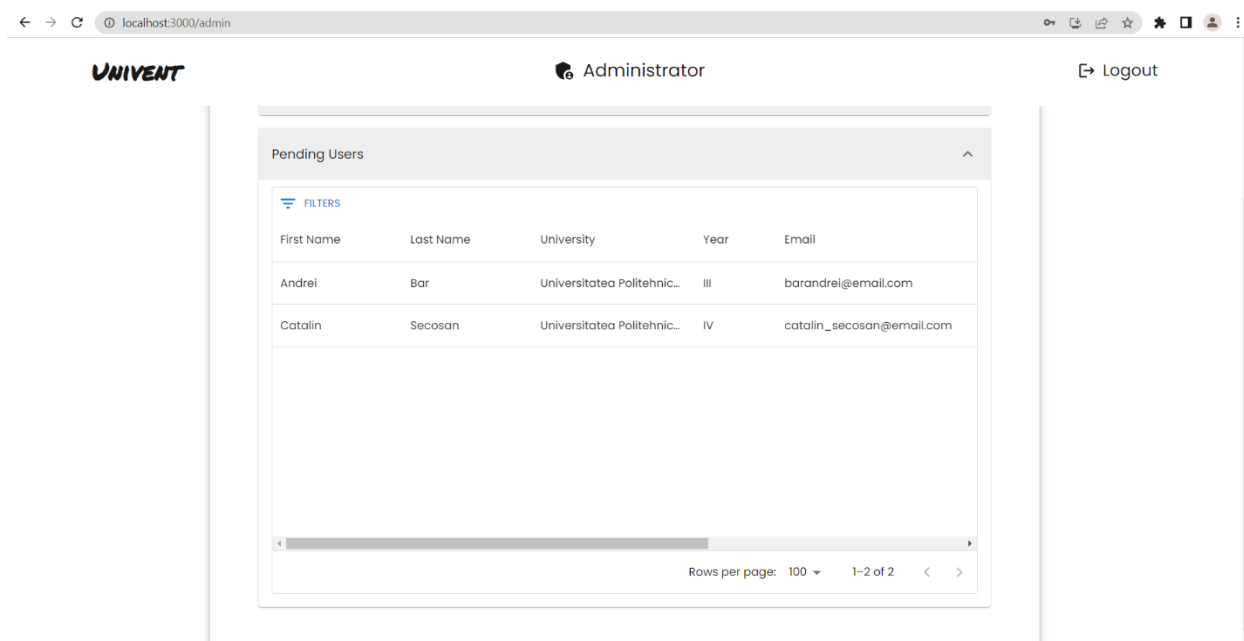


Figura 6.36: Lista conturilor noi de studenți

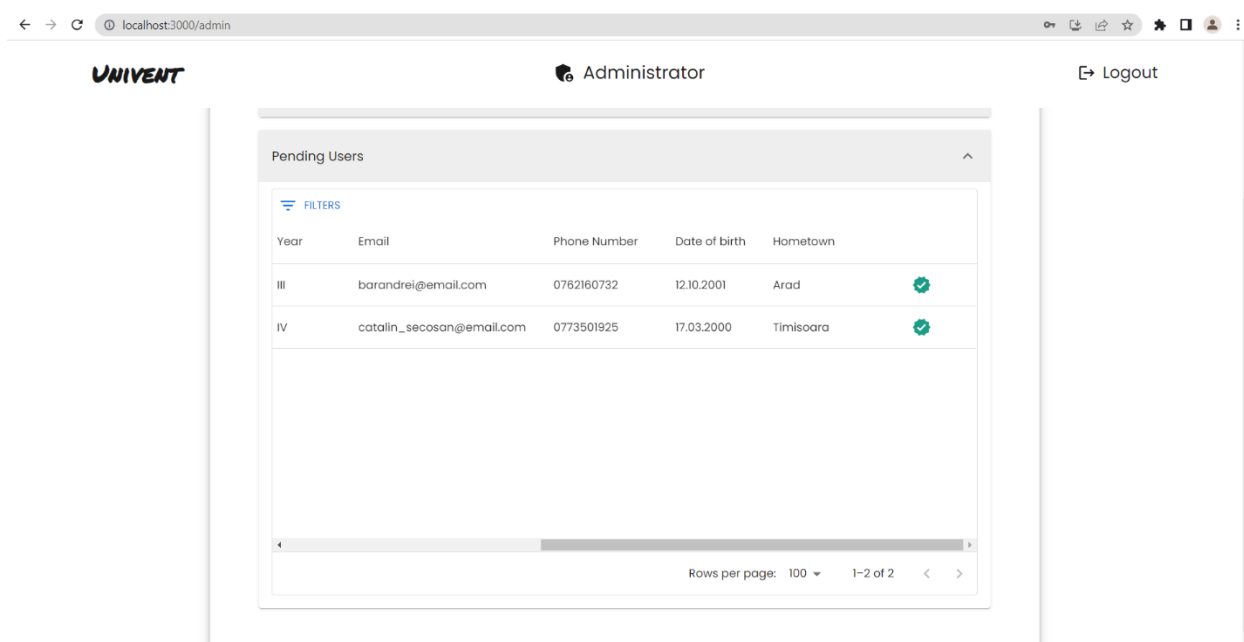


Figura 6.37: Opțiunea de aprobare utilizatorilor noi

7. CONCLUZII ȘI DIRECȚII DE CONTINUARE A DEZVOLTĂRII

În cadrul acestei lucrări, a fost proiectată și dezvoltată o aplicație web, ce a avut ca scop îmbunătățirea experienței studenților din cadrul mai multor universități. Această platformă le oferă tinerilor oportunitatea de a participa la evenimente extracurriculare și de a beneficia de experiențe unice pe perioada studiilor universitare. Astfel, aplicația UNIVENT este destinată tuturor studenților înrolați la una din universitățile partenere acestui proiect, având un rol esențial și în procesul de acomodare al studenților. Prin utilizarea acestei platforme, studenții pot socializa într-un mediu sigur, dezvoltându-și abilitățile de comunicare, de ascultare și de înțelegere a persoanelor din jur. De asemenea, tot prin interacțiunea cu alți studenți, tinerii au oportunitatea de a-și îmbogăți cunoștințele într-o varietate de domenii și de a experimenta o transformare în gândirea și percepția lor asupra vieții. Această interacțiune socială le permite să depășească cu succes provocările pe care le aduce viața universitară, deschizându-le noi perspective și oportunități valoroase.

După părerea mea, am reușit să implementez un sistem software complex și modern, cu numeroase cerințe funcționale și non-funcționale, ce este ușor și plăcut de utilizat. Consider această aplicație dedicată studenților, ca fiind captivantă și inovatoare, încurajându-i pe tineri să exploreze și să creeze evenimente personalizate, oferindu-le o platformă unică asemeni unei rețele de socializare. Am pus accent pe simplificarea procesului de organizare a evenimentelor și pe facilitarea interacțiunii între studenți, pentru o comunitate vibrantă și dinamică.

Platforma prezentată în această lucrare se află într-un stadiu matur al dezvoltării, însă tehnologia evoluează în permanență. Astfel, pentru următoarele versiuni, aplicația UNIVENT poate fi îmbunătățită și extinsă prin adăugarea de funcționalități noi și optimizarea celor existente. Câteva exemple de astfel de îmbunătățiri, ar fi:

- Implementarea unui client mobil, pentru sistemele de operare Android și iOS;
- Optimizarea performanței endpoint-urilor, pe partea de server;
- Automatizarea procesului de verificare a informațiilor și aprobare a utilizatorilor noi, de către administrator;
- Utilizarea platformelor Cloud pentru a stoca toate imaginile utilizate în cadrul platformei, într-un mod securizat;
- Integrarea platformei cu un sistem specializat în „deployment”, precum Microsoft Azure sau „AWS” (Amazon Web Services);
- Îmbunătățirea procesului de configurare a sistemului, prin intermediul platformelor dedicate, cum ar fi „Docker”;
- Posibilitatea de a accesa profilul altui student, pentru a afla mai multe informații;
- Adăugarea unui sistem de mesagerie instantanee;
- Adăugarea unui serviciu de apelare a organizatorului unui eveniment, în caz de întrebări, nelămuriri sau sugestii;
- Adăugarea de notificări în cazul anulării sau actualizării informațiilor evenimentelor planificate;
- Implementarea unui sistem pentru trimiterea informațiilor și noutăților prin e-mail.

REFERINȚE BIBLIOGRAFICE

- [1] Brodsky, I., *The History of Wireless: How Creative Minds Produced Technology for the Masses*. Telescope Books, 2008.
- [2] *** „Brief History of the Internet”. Disponibil online: <https://www.internetsociety.org/resources/doc/2017/brief-history-internet/>. (accesat aprilie 2023)
- [3] *** „Cine Este Tim Berners-Lee?”. Disponibil online: <https://ro.eyewater.com/cine-este-tim-berners-lee/>. (accesat aprilie 2023)
- [4] *** „Ce este HTTP (Hypertext Transfer Protocol)?”. Disponibil online: <https://blog.hostx.ro/utile/ce-este-http-hypertext-transfer-protocol/>. (accesat aprilie 2023)
- [5] *** „HTTP Messages”. Disponibil online: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages>. (accesat aprilie 2023)
- [6] *** „HTTP response status codes”. Disponibil online: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status#information_responses. (accesat aprilie 2023)
- [7] *** „Structura URL”. Disponibil online: [https://www.theseo.ro/structura-url-cum-sa-creati-o-adresa-bine-structurata-potrivita-in-seo/#Structura URL a website-ului](https://www.theseo.ro/structura-url-cum-sa-creati-o-adresa-bine-structurata-potrivita-in-seo/#Structura_URL_a_website-ului). (accesat aprilie 2023)
- [8] *** „Ce este un URL?”. Disponibil online: <https://dwf.ro/blog/ce-este-un-url/>. (accesat aprilie 2023)
- [9] *** „HTTP vs HTTPS: Comparison, Pros and Cons, and More”. Disponibil online: [https://www.hostinger.com/tutorials/http-vs-https#Differences Between HTTP vs HTTPS](https://www.hostinger.com/tutorials/http-vs-https#Differences_Between_HTTP_vs_HTTPS). (accesat aprilie 2023)
- [10] *** „Facebook launches standalone “Events” discovery and calendar app”. Disponibil online: <http://tcn.ch/2dRO0w2>. (accesat mai 2023)
- [11] *** „Overview of ASP.NET Core”. Disponibil online: <https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core?view=aspnetcore-6.0>. (accesat ianuarie 2023)
- [12] *** „What is REST”. Disponibil online: <https://restfulapi.net/>. (accesat ianuarie 2023)
- [13] Sfetcu, N., *Proiectarea, dezvoltarea, și întreținerea siturilor web*. MultiMedia Publishing, 2018.
- [14] *** „SQL tools overview”. Disponibil online: <https://learn.microsoft.com/en-us/sql/tools/overview-sql-tools?view=sql-server-ver16>. (accesat ianuarie 2023)
- [15] Banks, A., Porcello, E., *Learning React: Functional Web Development with React and Redux*. Prima ediție, O'Reilly Media, 2017.
- [16] *** „Client-Server Architecture – Definition, Types, Examples, Advantages & Disadvantages”. Disponibil online: <https://www.thecrazyprogrammer.com/2021/03/client-server-architecture.html>. (accesat februarie 2023)
- [17] Martin, R., *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Pearson, 2017.

- [18] *** „Design Pattern - Factory Pattern”. Disponibil online:
https://www.tutorialspoint.com/design_pattern/factory_pattern.htm.
(accesat ianuarie 2023)
- [19] *** „CQRS pattern”. Disponibil online:
<https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>.
(accesat ianuarie 2023)
- [20] *** „JSON web token | JWT”. Disponibil online: <https://www.geeksforgeeks.org/json-web-token-jwt/>. (accesat februarie 2023)
- [21] *** „Hashing in Action: Understanding bcrypt”. Disponibil online:
<https://auth0.com/blog/hashing-in-action-understanding-bcrypt/>. (accesat mai 2023)
- [22] *** „React Components - Material UI”. Disponibil online: <https://mui.com/material-ui/>.
(accesat martie 2023)

DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR*

Subsemnatul SECOȘAN CĂTĂLIN-LUCA

legitimat cu C.I. seria T2 nr. 562737
CNP 5001219350024
autorul lucrării UNIVENT - APLICAȚIE WEB PENTRU STUDENȚI

elaborată în vederea susținerii examenului de finalizare a studiilor de LICENȚĂ organizat de către Facultatea AUTOMATICĂ ȘI CALCULATOARE din cadrul Universității Politehnica Timișoara, sesiunea Iunie a anului universitar 2022-2023, coordonator CONF. DR. ING. ALEXANDRU TOPÎRCEANU luând în considerare art. 34 din Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării ~~am utilizat instrumente specifice inteligenței artificiale (IA) și anume~~ (denumirea) (sursa), ~~pe care le-am citat în conținutul lucrării/nu am utilizat~~ instrumente specifice inteligenței artificiale (IA)¹.

Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

23.06.2023

Semnătura

Secosom

*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

¹ Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA